🛡️ Module 11: Reverse Engineering, Fuzzing and Binary Exploitation

```
# Exercise 01: Performing Binary Analysis on Linux
```

Scenario

Binary Analysis on Linux involves reverse engineering compiled executable binaries to understand their control flow, data structures, internal functions, and security protections (such as endianness, DEP/NX, and ASLR). When source code is unavailable, penetration testers and reverse engineers utilize static analysis tools (like `file`, `strings`, `rabin2`, `xxd`, `readelf`, and `objdump`) and dynamic debuggers (like `gdb`, `radare2`, and `edb`) to inspect the ELF (Executable and Linkable Format) header structure, disassemble machine instructions, set breakpoints, and extract embedded secrets or authentication logic.

In this exercise, we log into the **Software-Test-Linux-32bit** machine (**student** / **studentpassword**), inspect a 32-bit ELF CTF binary (**crackme0x00a**), analyze binary headers and embedded data strings using **file**, **rabin2 -I**, **strings**, **xxd**, and **rabin2 -z**, perform disassembly with **Radare2** (**pdf @ main**) and **GDB** (**disassemble main**), set a breakpoint at the string comparison function (`strcmp` at **0x0804852a**), inspect runtime memory to recover the hardcoded password (**g00dJ0B!**), launch **Evan's Debugger** (`edb`), analyze AT&T vs. Intel assembly syntax differences via **objdump** (**objdump -d -M intel /bin/bash**), inspect CPU endianness via `lscpu`, `/proc/cpuinfo`, and GDB register inspection (**info registers**), craft a 32-bit x86 assembly program (**code-one.asm**) using Linux system calls (`sys_write` and `sys_exit`), assemble and link it using **NASM** and **ld**, and audit ELF headers, program segments, section headers, symbol tables, and relocated bytes via **readelf** (**readelf -h**, **readelf -l**, **readelf -S**, **readelf -s**, and **readelf -x .text**).

Objective

The objective of this lab is to help students learn how to:
- Authenticate to 32-bit Linux test environment (**student** / **studentpassword**)
- Conduct static binary identification using **file crackme0x00a** and **rabin2 -I crackme0x00a**
- Inspect binary strings and raw hex dumps using **strings**, **xxd**, and **rabin2 -z crackme0x00a**
- Perform disassembly and control flow analysis in Radare2 using **radare2** and **pdf @ main**
- Perform dynamic debugging in GDB using **gdb crackme0x00a**, set breakpoints at `strcmp` (**break *0x0804852a**), and inspect compare registers to recover the hardcoded password: **g00dJ0B!**
- Explore GUI debugging interface via Evan's Debugger (**edb**)
- Compare AT&T vs. Intel assembly syntax formatting via **objdump -d /bin/bash** and **objdump -d -M intel /bin/bash**
- Determine system architecture details and Little-Endian memory order using **lscpu**, **cat /proc/cpuinfo**, and GDB **info registers**
- Author x86 32-bit assembly code (**code-one.asm**) using `int 0x80` system calls, assemble via **nasm -f elf32 -o code-one.o code-one.asm**, link via **ld -o code-one code-one.o**, and execute
- Perform comprehensive ELF binary header and section inspection using **readelf -h**, **readelf -l**, **readelf -S**, **readelf -s**, and **readelf -x .text**

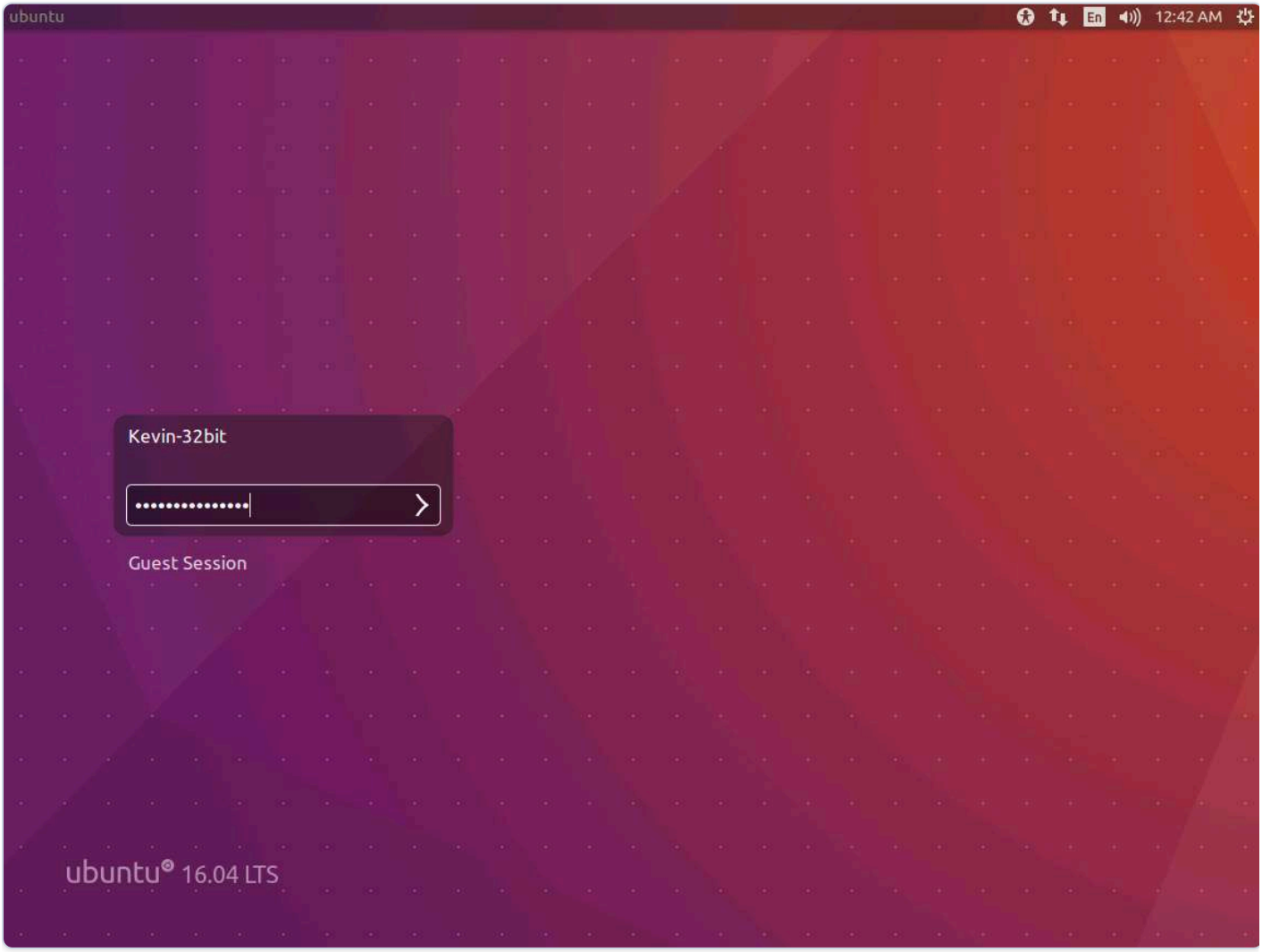
Task 1: Perform Binary Analysis on Linux

In this task, we perform static and dynamic analysis on `crackme0x00a` on the 32-bit Linux VM, locate the hardcoded password (`g00dJ0B!`) using GDB, explore GUI debugger `edb`, compare AT&T vs. Intel assembly syntax, write a

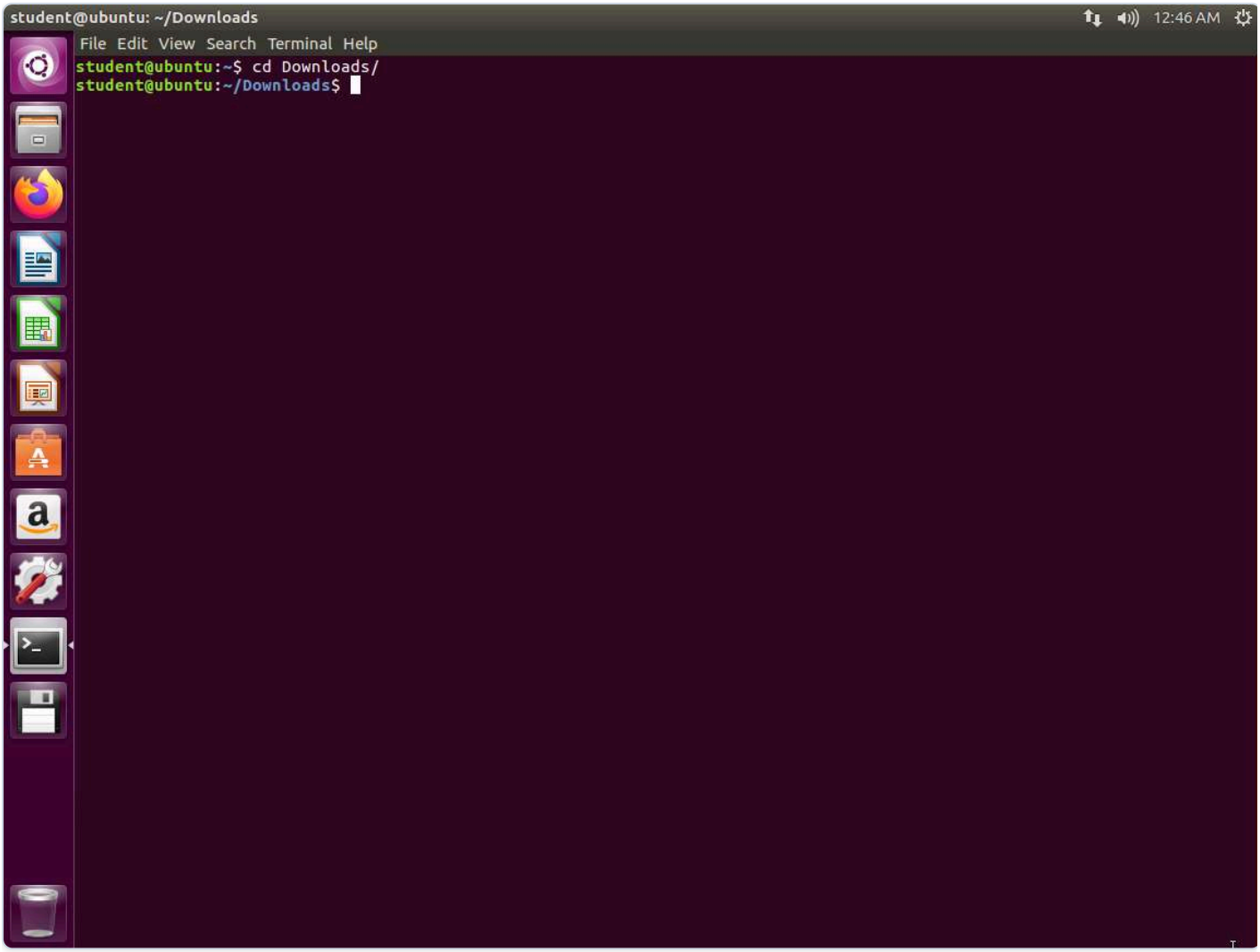
custom 32-bit assembly program (`code-one.asm`), and inspect ELF headers using `readelf` .

Lab Tasks

1. Log into **Software-Test-Linux-32bit** VM using credentials (**student** / **studentpassword**).

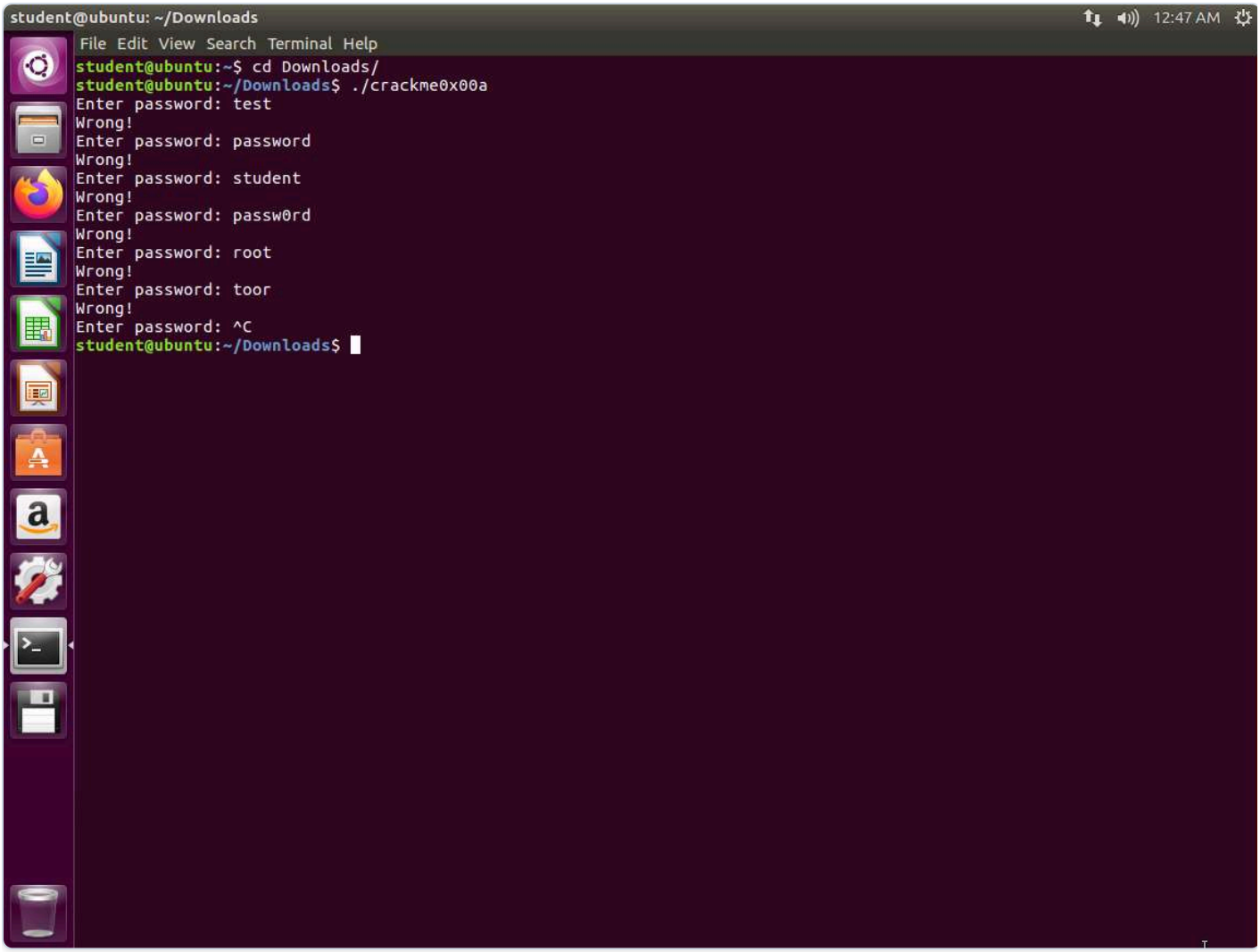


1. Open terminal and navigate to Downloads directory:
   cd Downloads



1. Attempt to run the target binary and enter test password inputs:

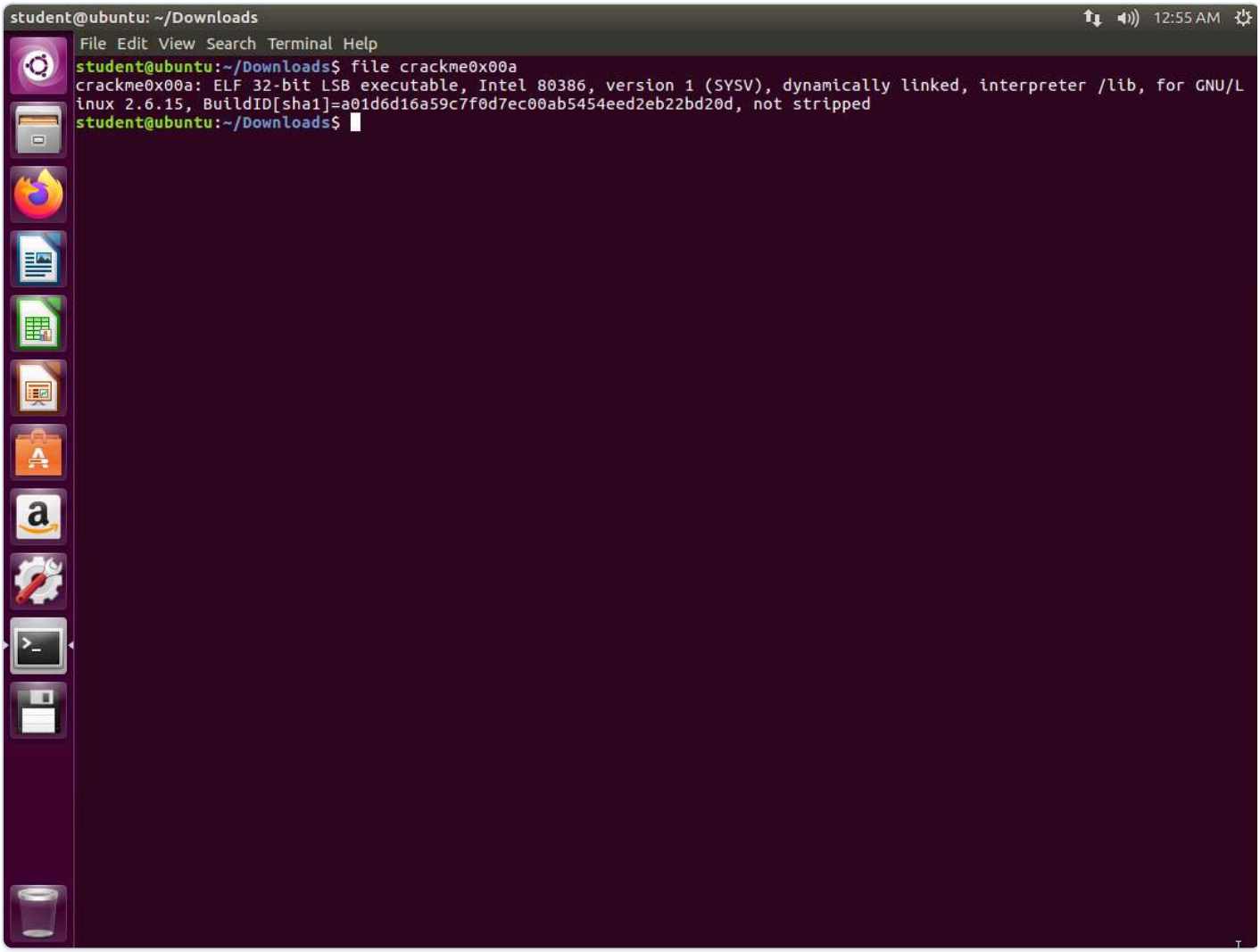
   ./crackme0x00a



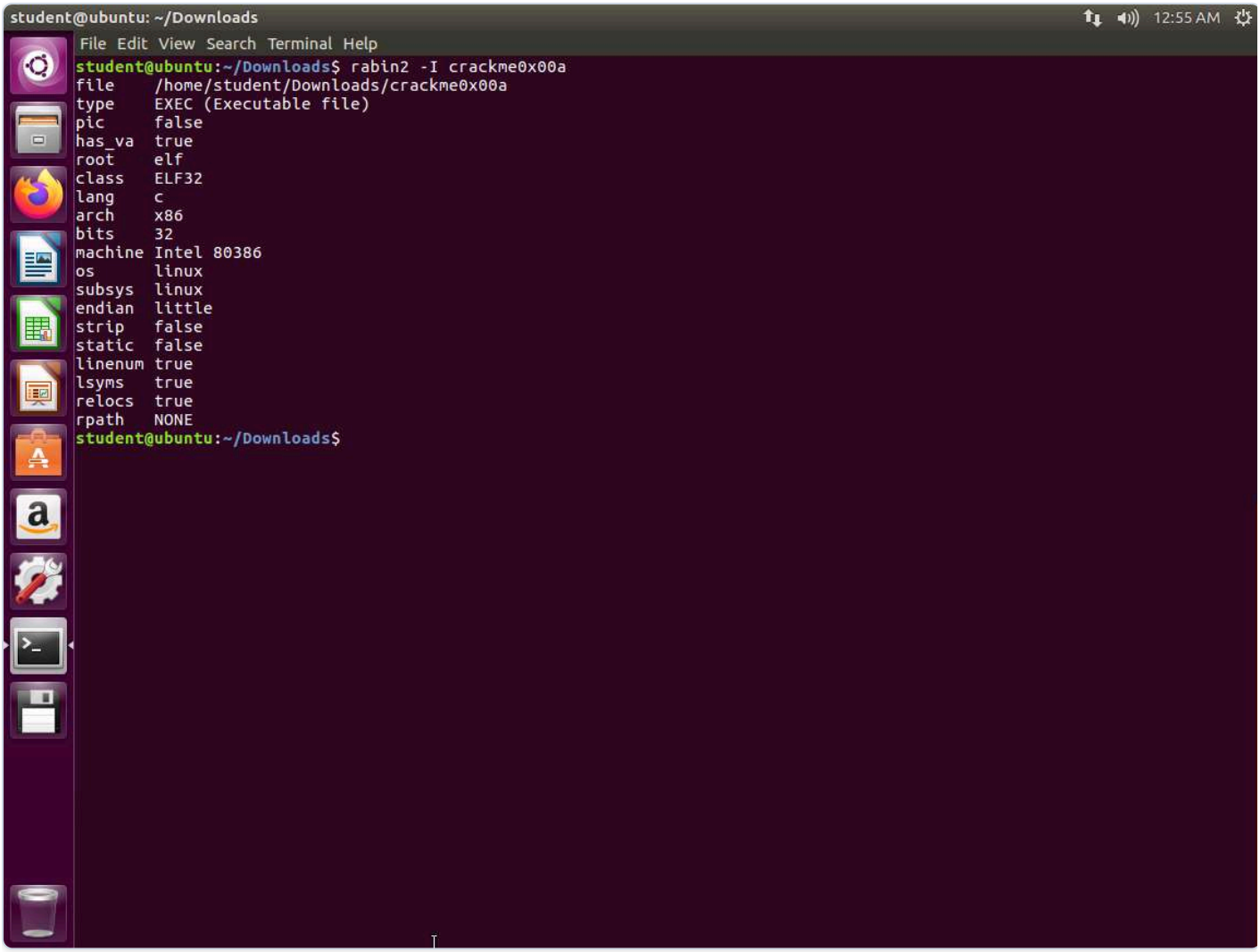
1. Determine file type and architecture details using `file`:

   file crackme0x00a

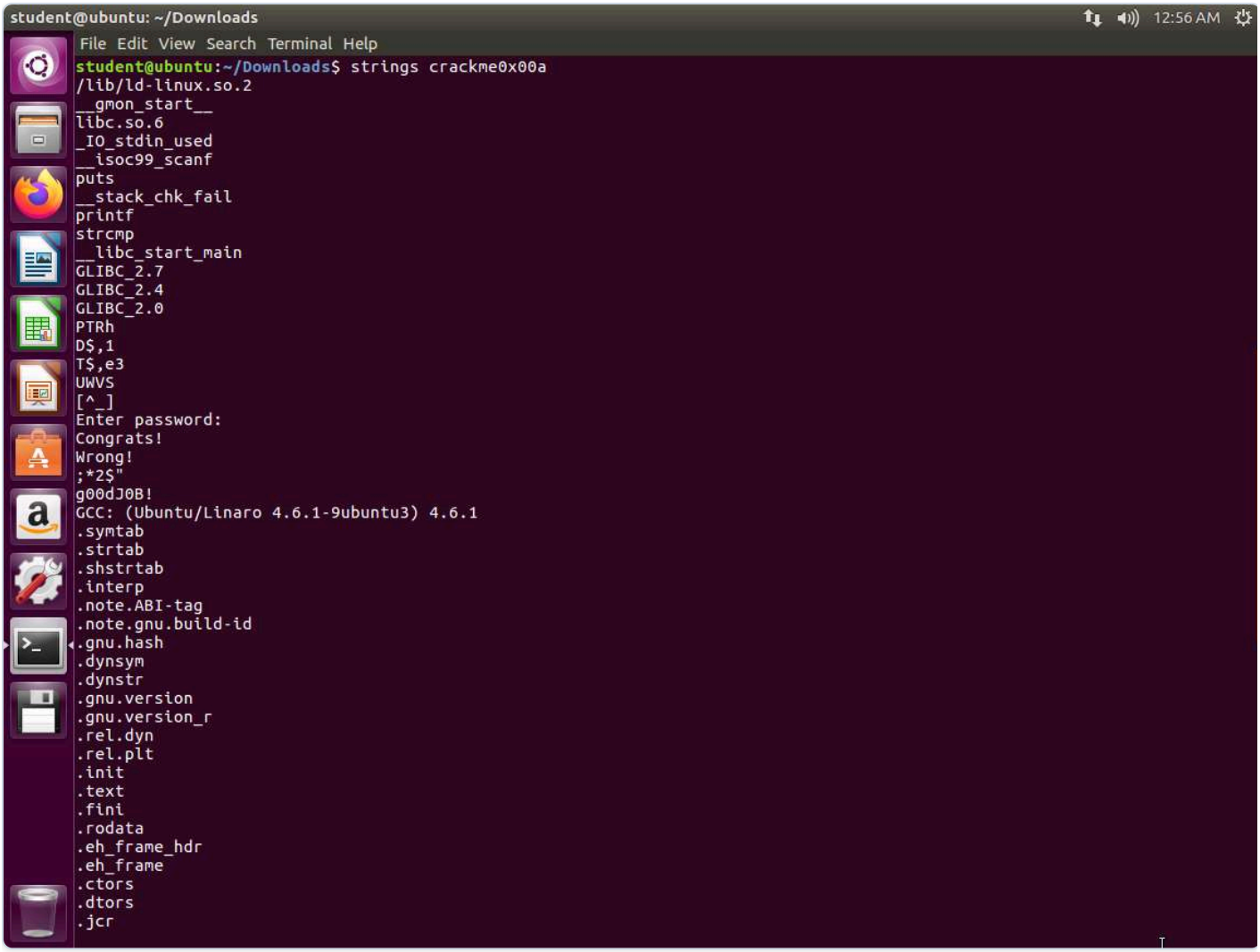
   Observe 32-bit LSB executable ELF format.



1. Inspect binary properties using Radare2 utility `rabin2`:
   rabin2 -I crackme0x00a

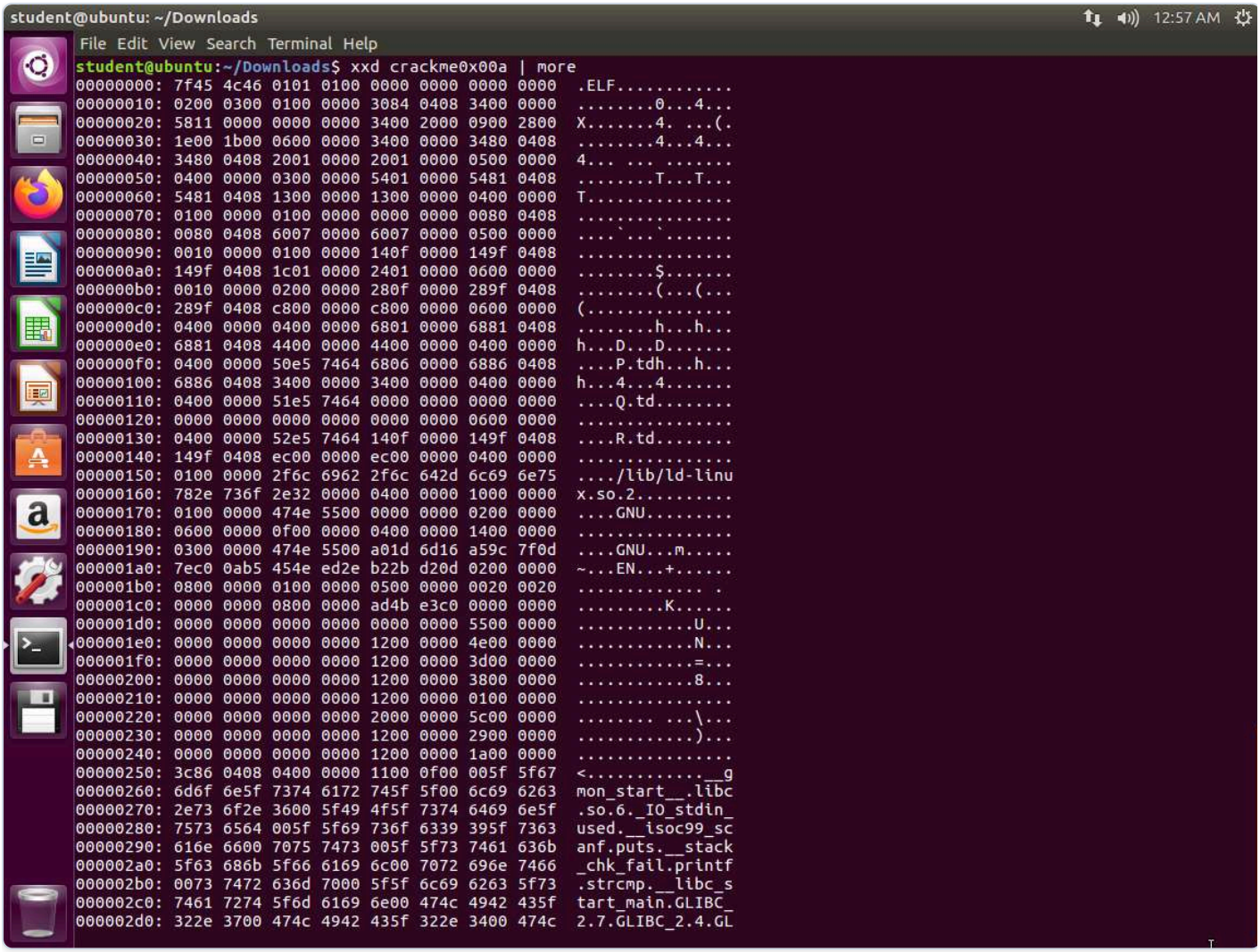


1. Extract printable ASCII strings embedded in the binary:
   strings crackme0x00a

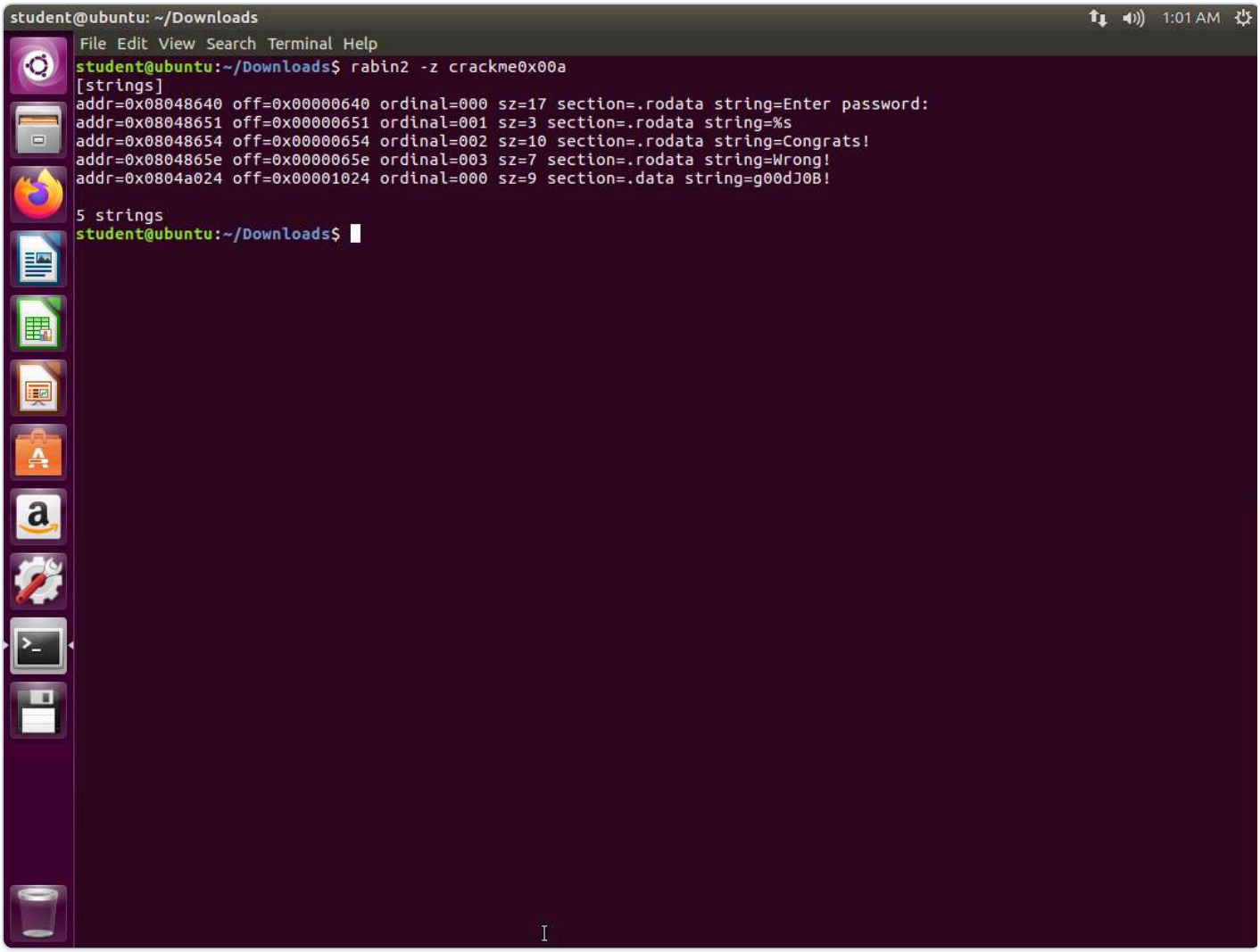


1. View raw hexadecimal structure using `xxd`:

   xxd crackme0x00a | more



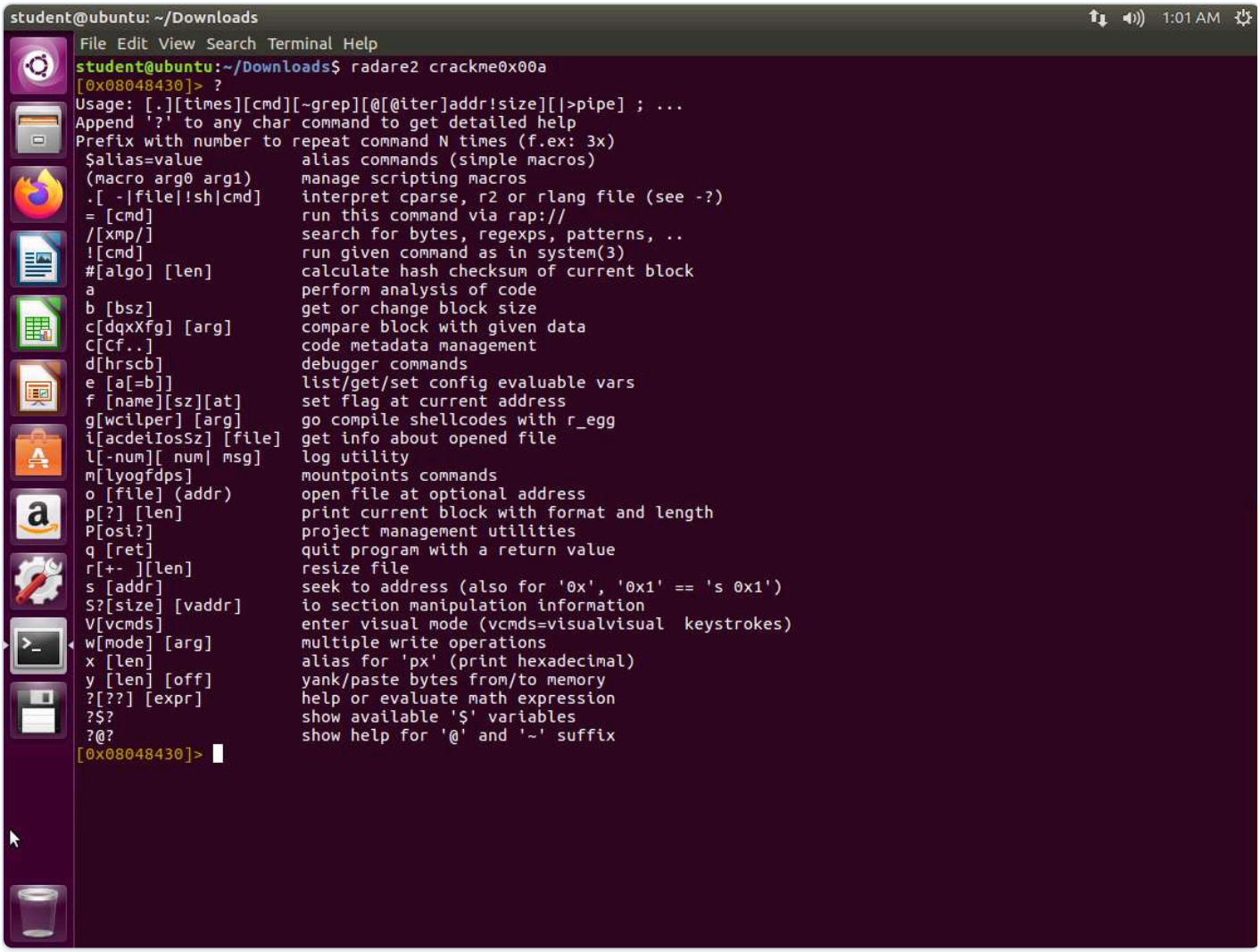
1. Extract strings specifically from the `.data` section using `rabin2`:
   rabin2 -z crackme0x00a



1. Open binary in Radare2 framework and display help menu:

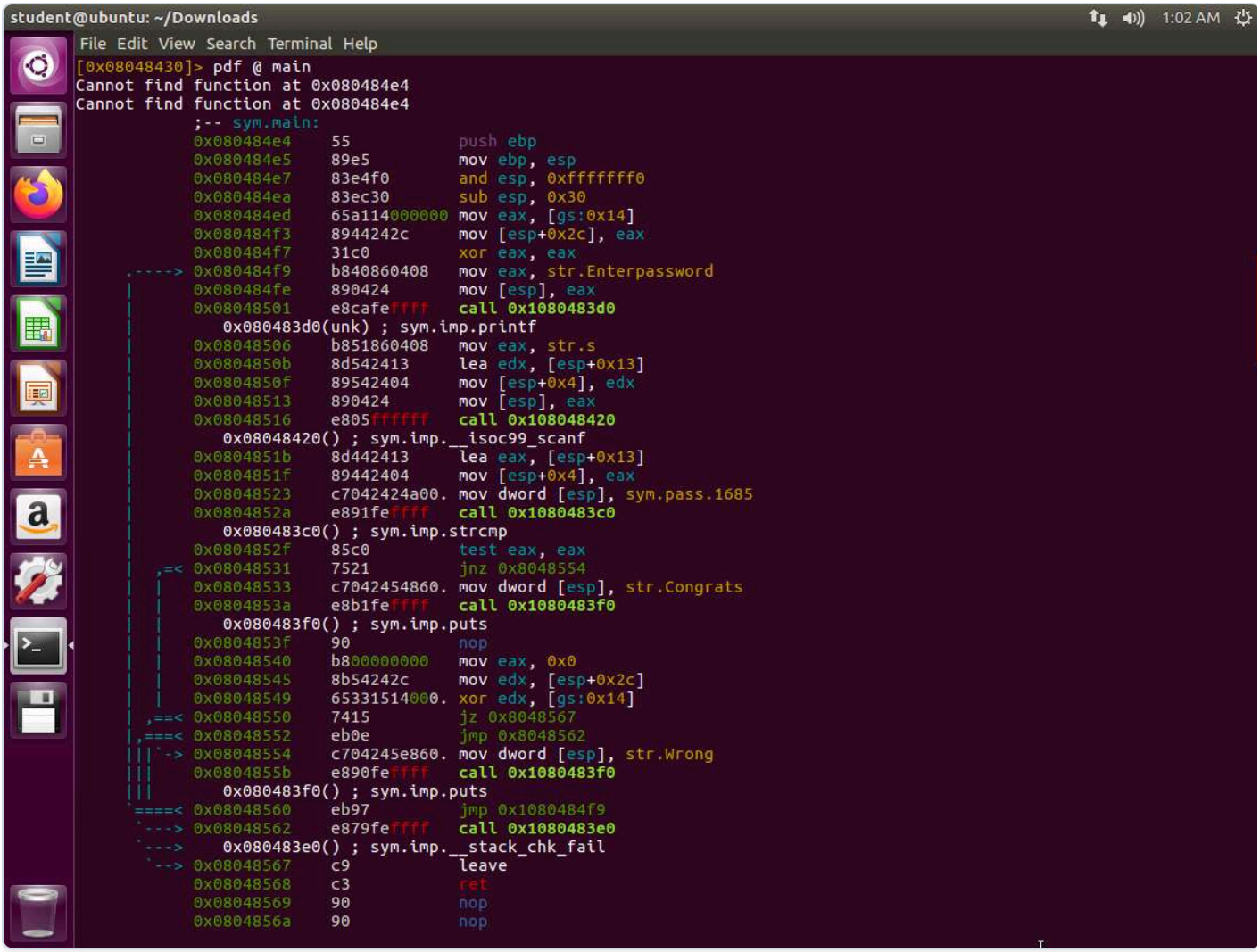
   radare2 crackme0x00a

   ?

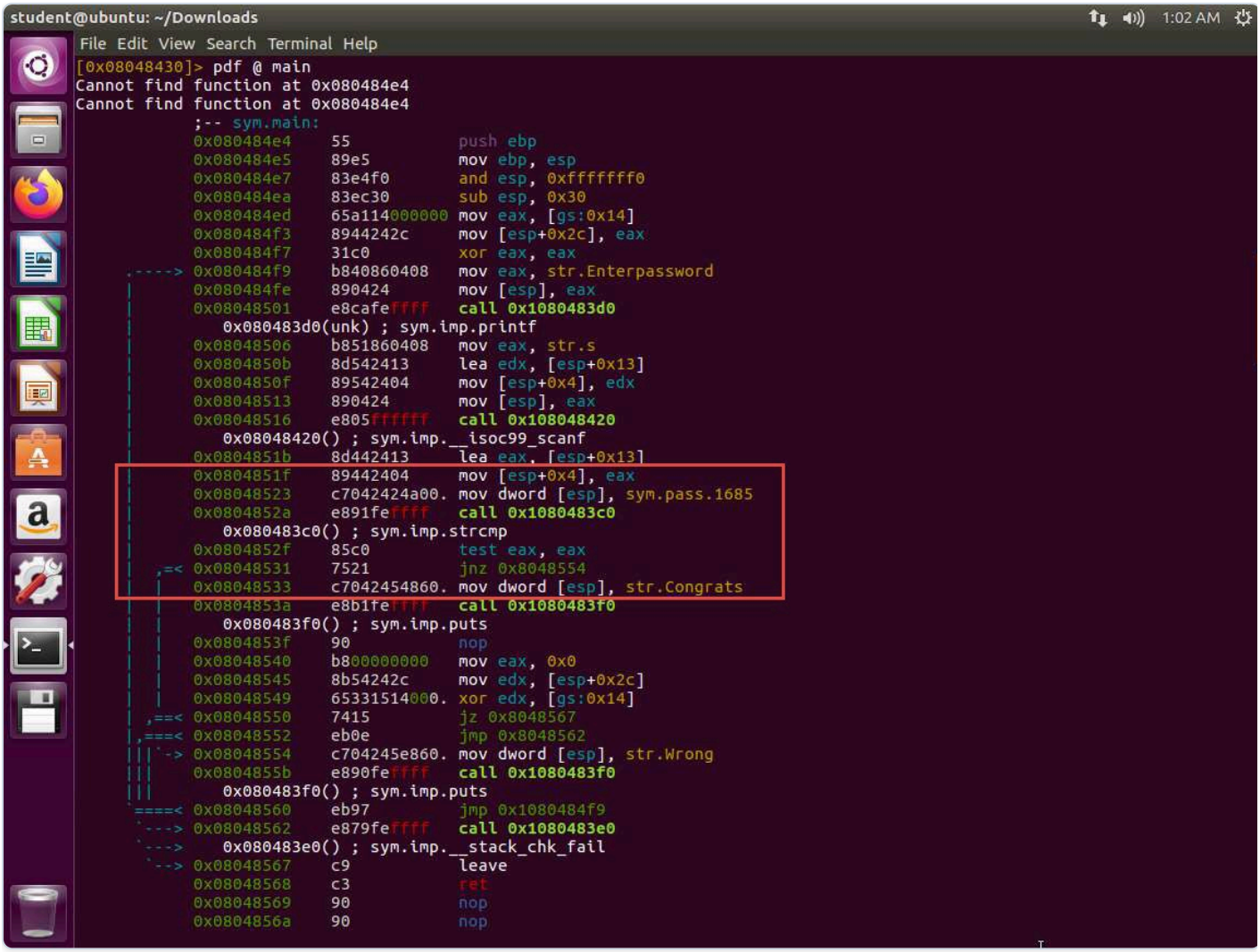


1. Disassemble main function inside Radare2:

   pdf @ main



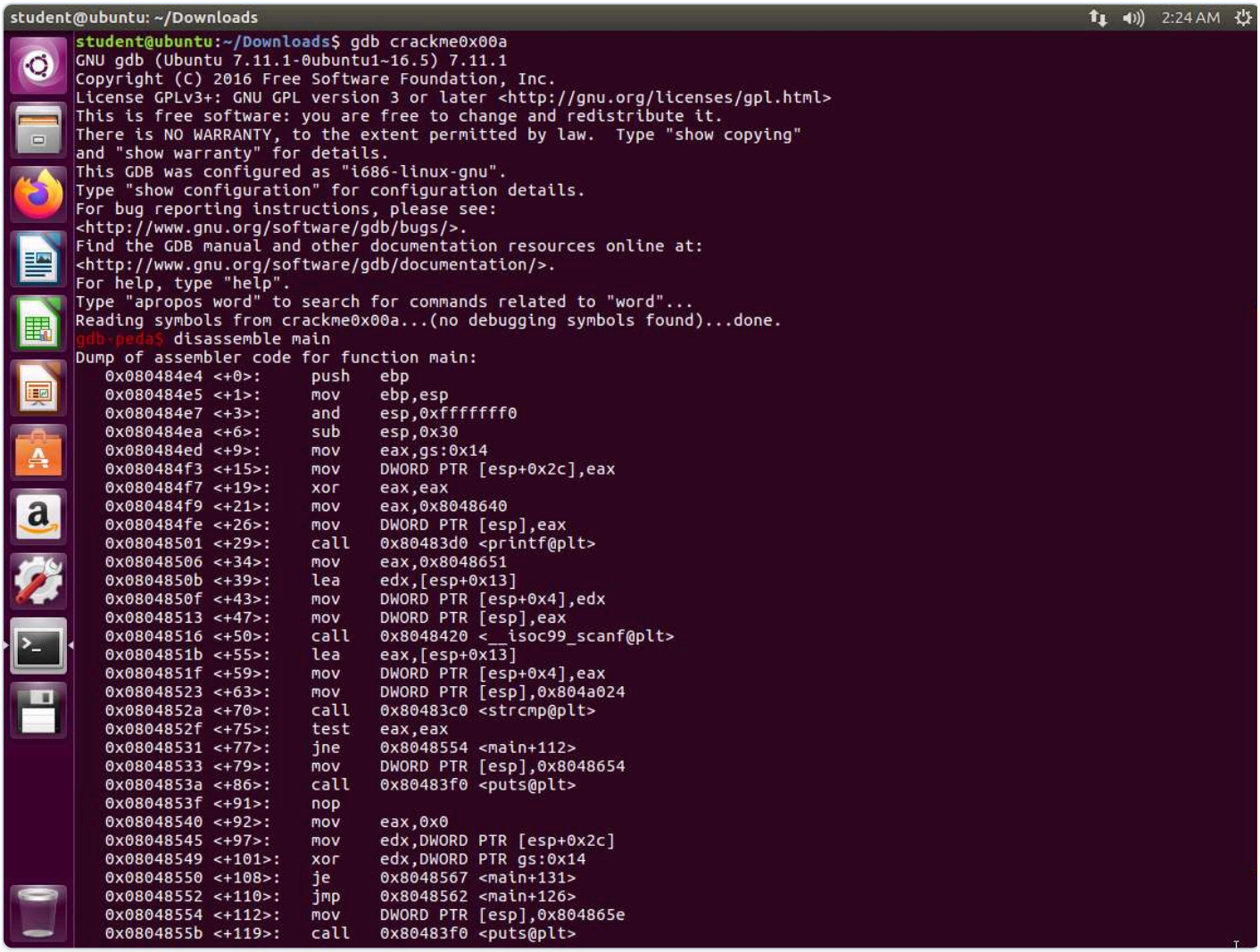
1. Locate string comparison function `strcmp` within disassembled instructions.



1. Exit Radare2, launch GDB, and disassemble main function:

   gdb crackme0x00a

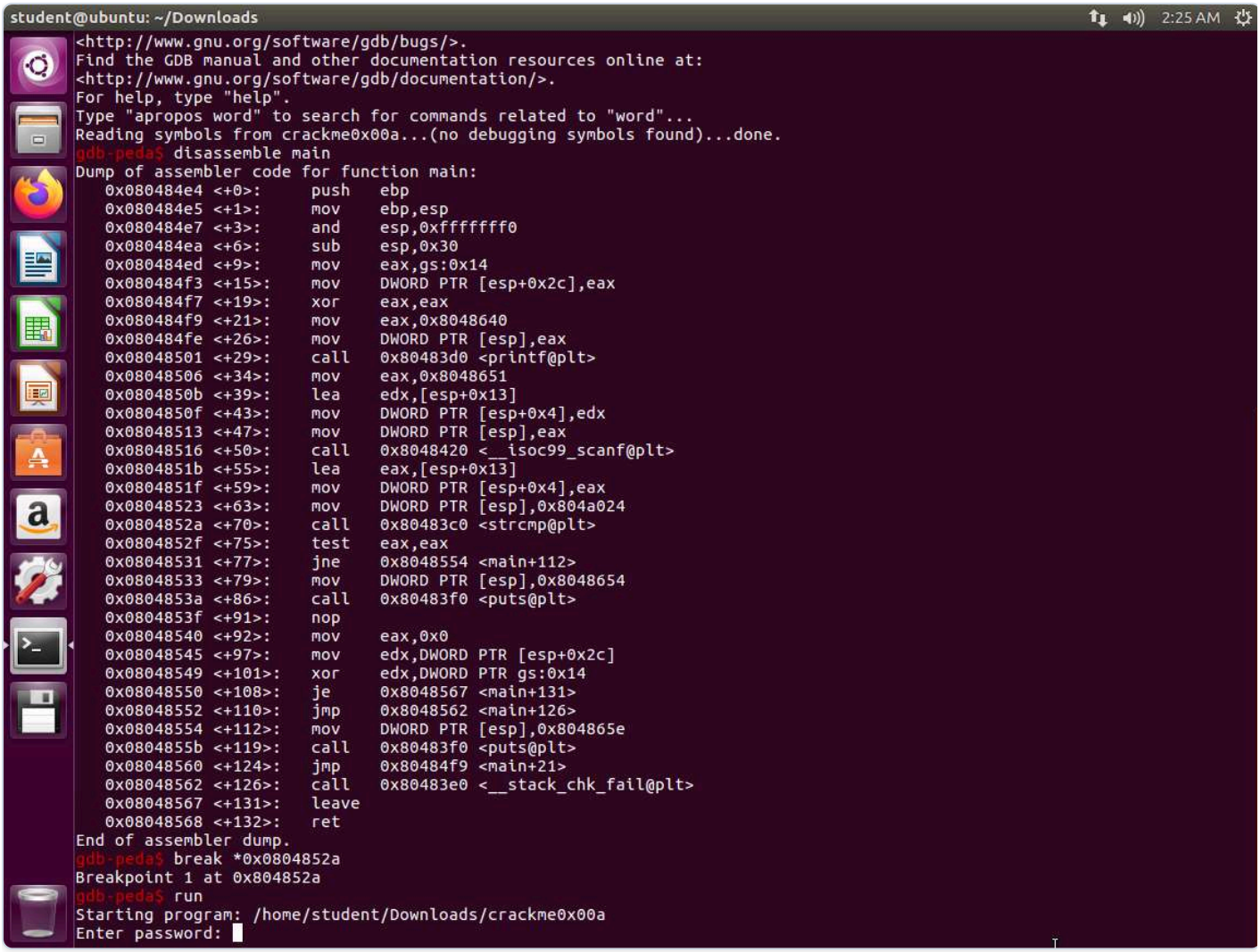
   disassemble main



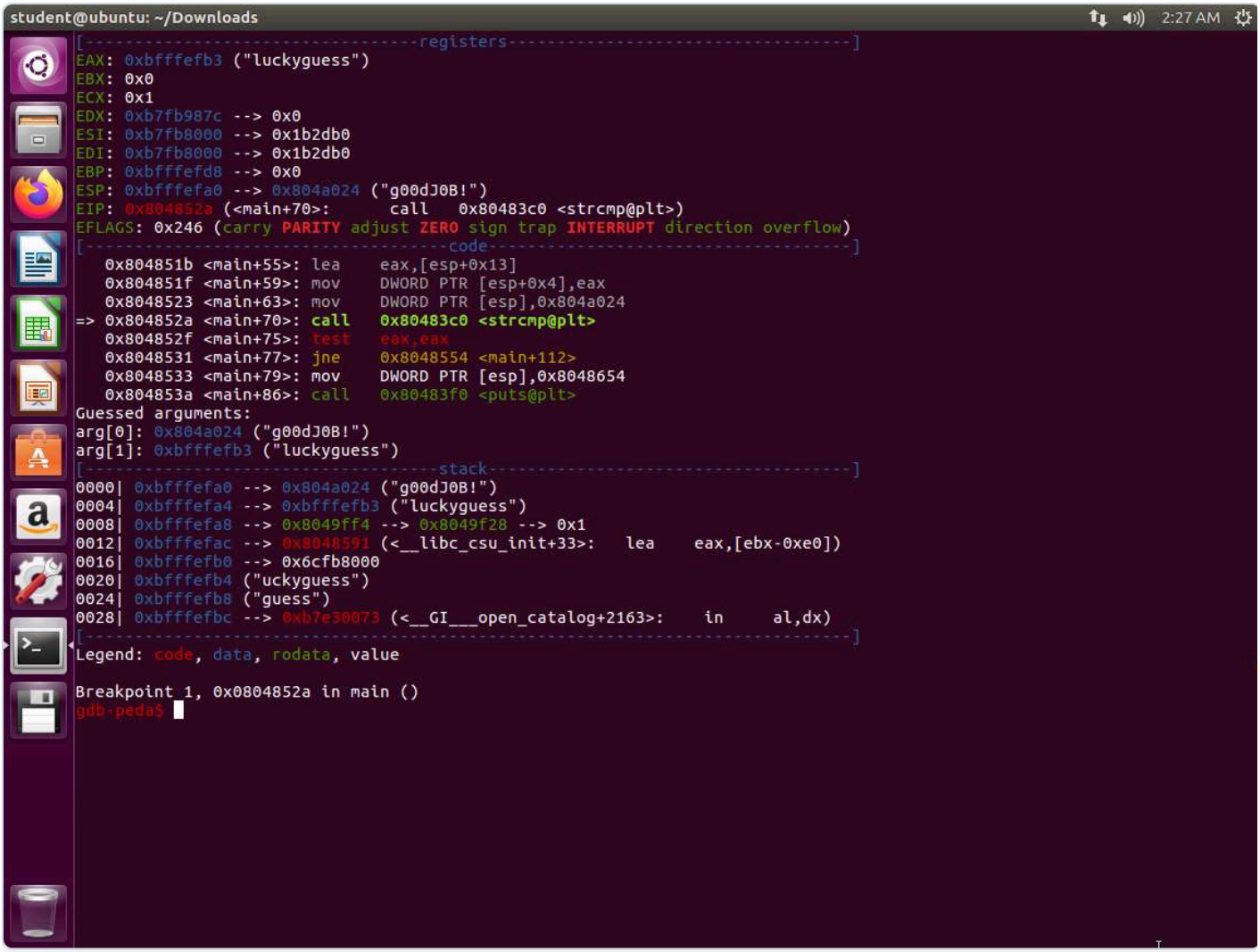
1. Set breakpoint right before `strcmp` instruction at address **0x0804852a** and run execution:

   break *0x0804852a

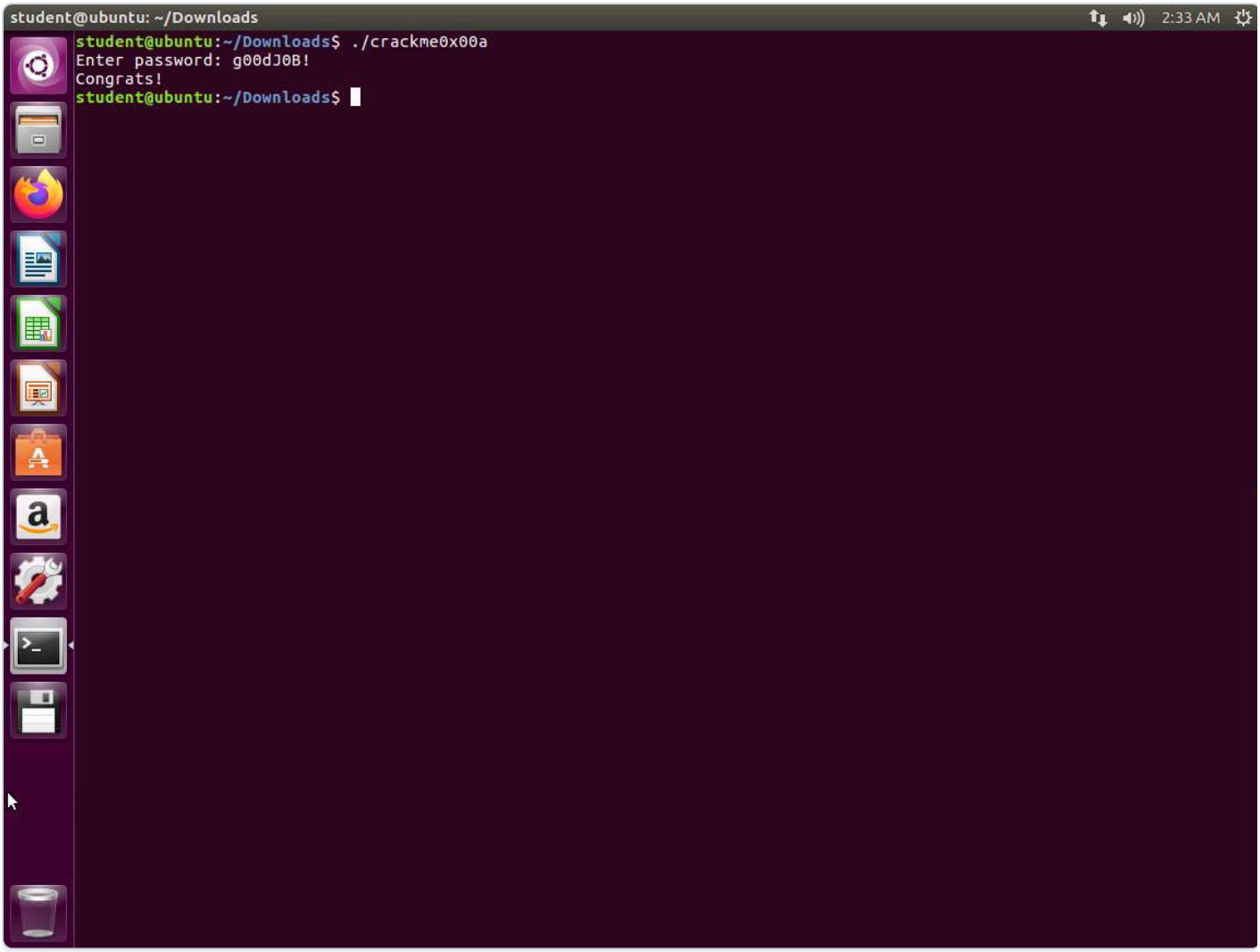
   run



1. Enter test string `luckyguess` and inspect memory comparison registers to reveal actual password (**g00dJ0B!**).

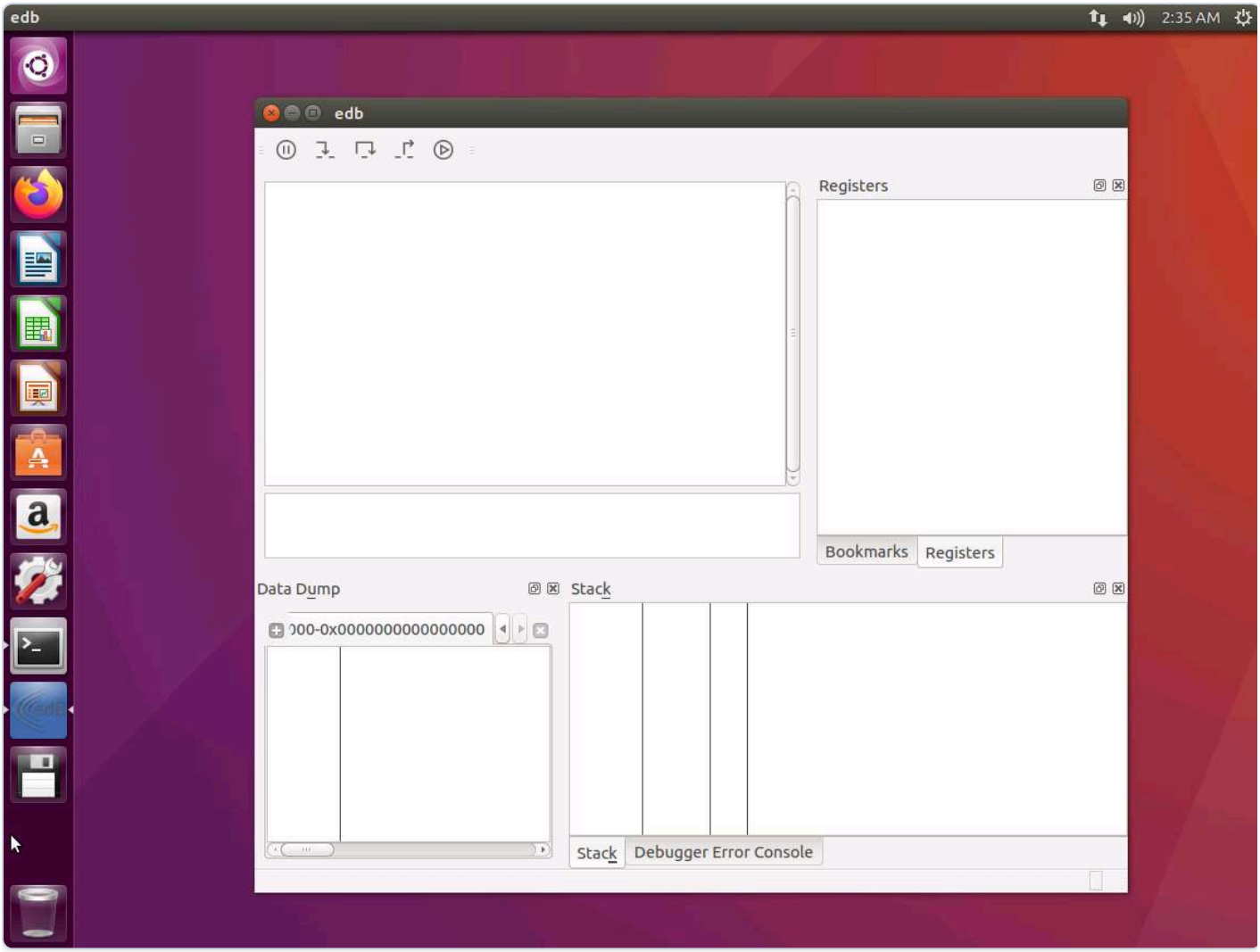


1. Test the recovered password against the binary to confirm successful authentication:
   ./crackme0x00a -> Password: **g00dJ0B!**

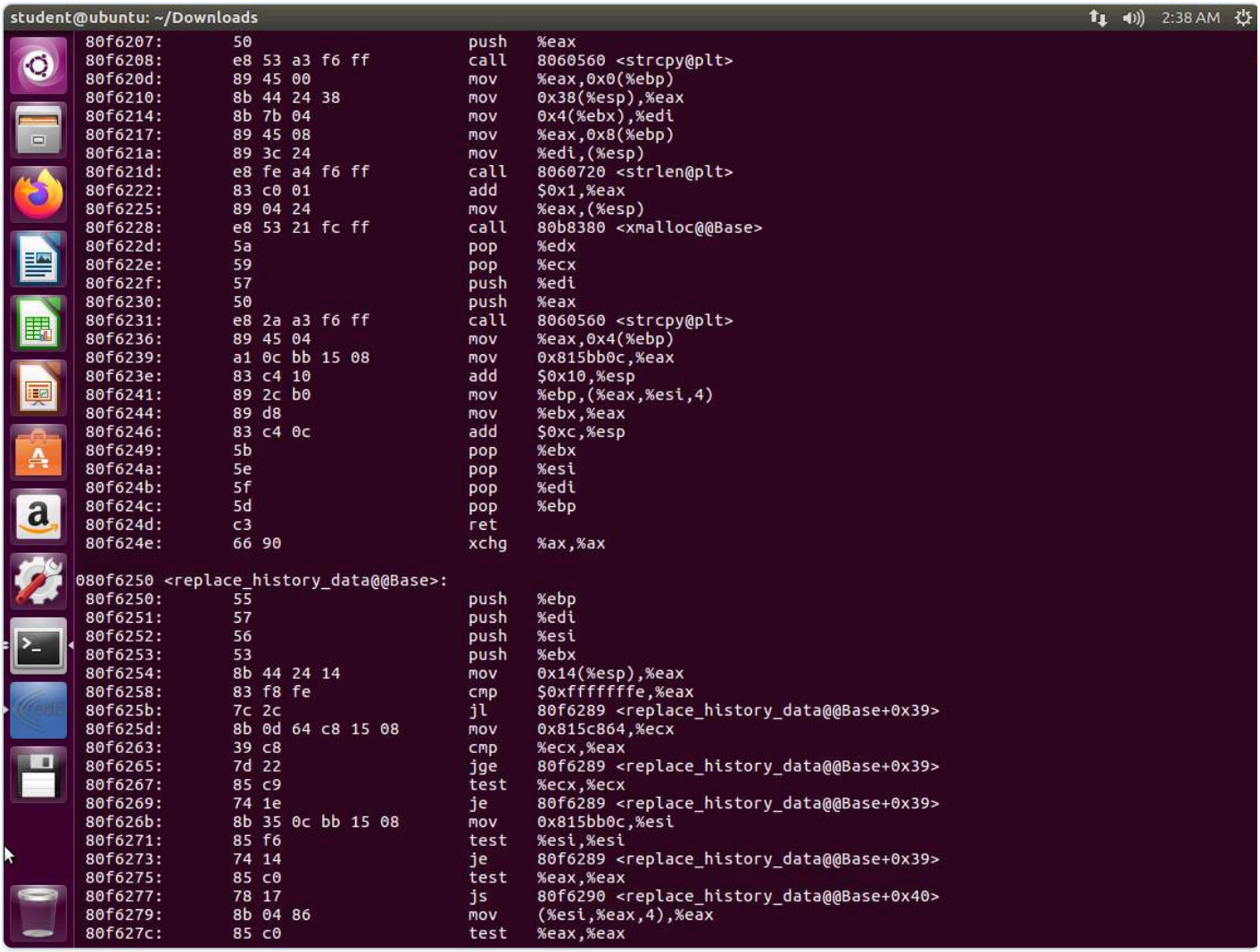


1. Launch Evan's Debugger GUI interface:

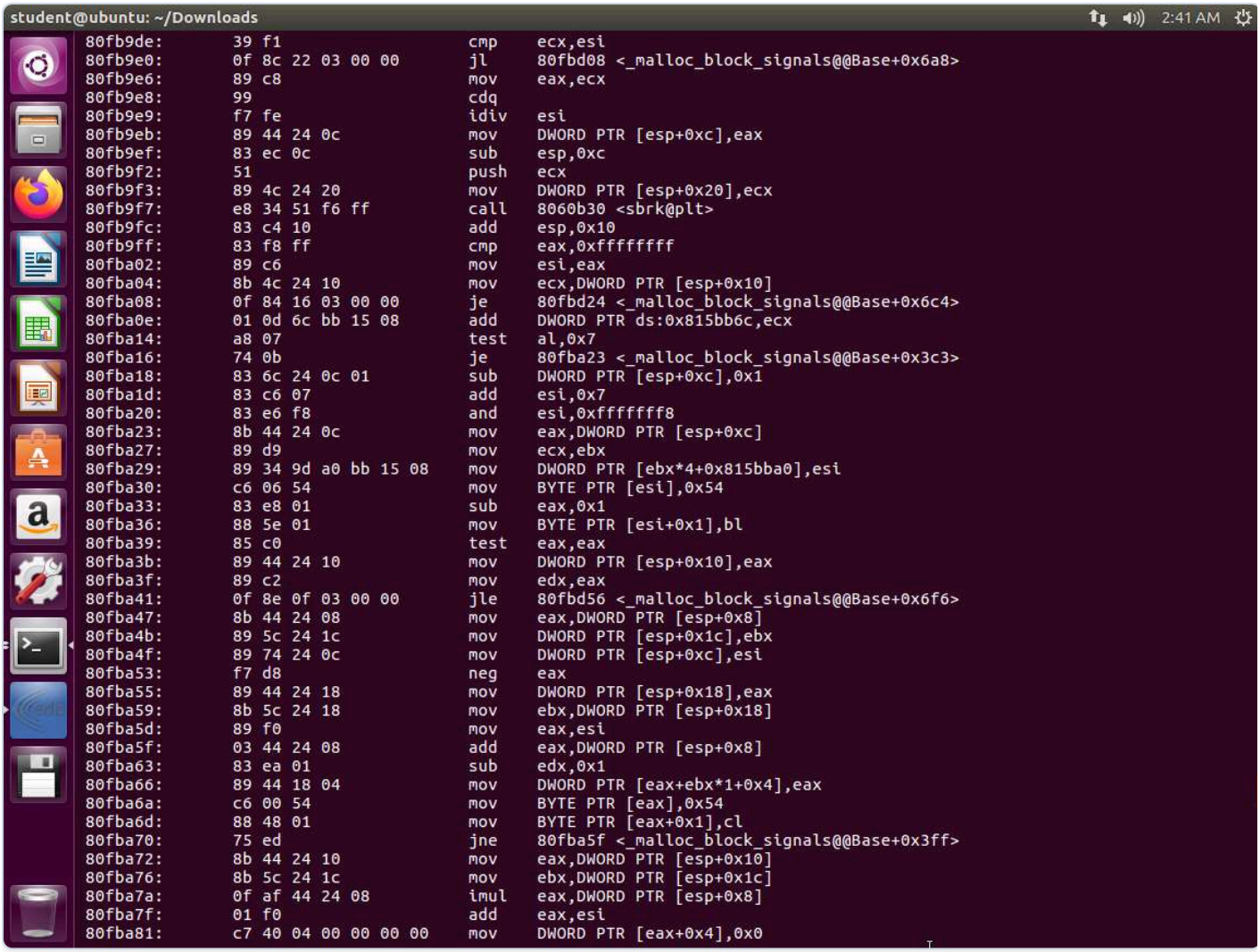
   edb



1. Disassemble `/bin/bash` binary using standard AT&T syntax:
   objdump -d /bin/bash

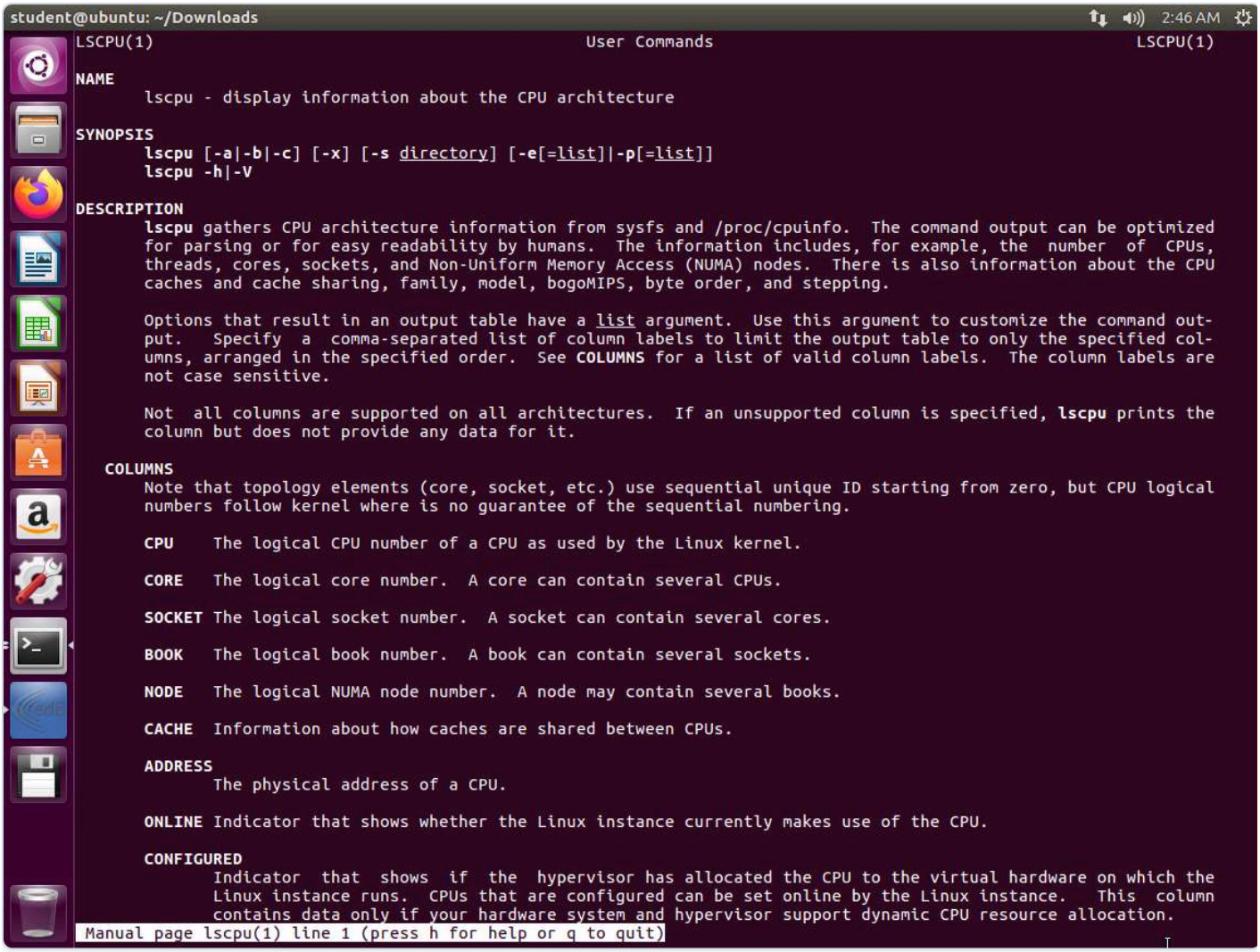


1. Disassemble `/bin/bash` binary using Intel assembly syntax:
   objdump -d -M intel /bin/bash



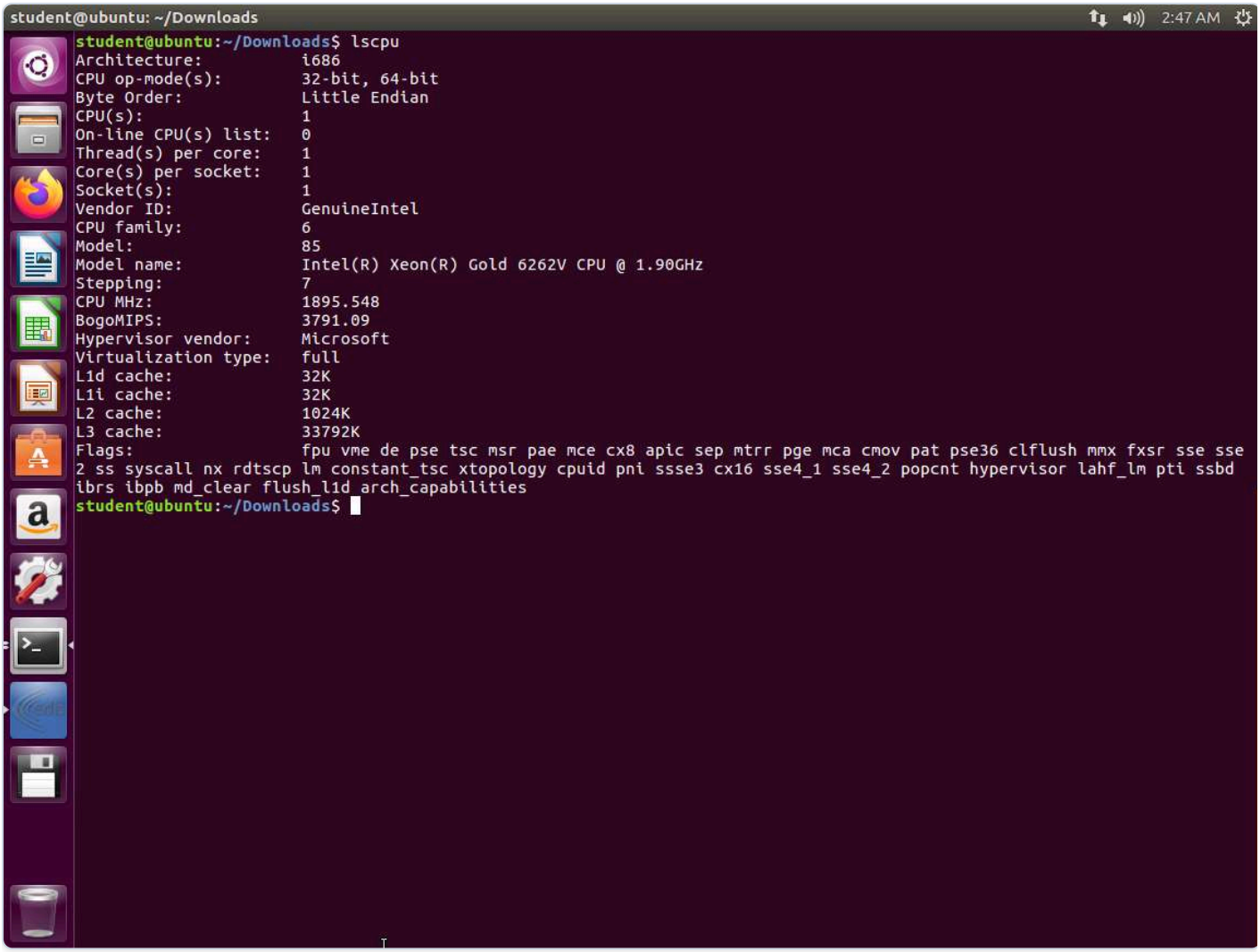
1. Open manual page for CPU architecture utility:

   man lscpu



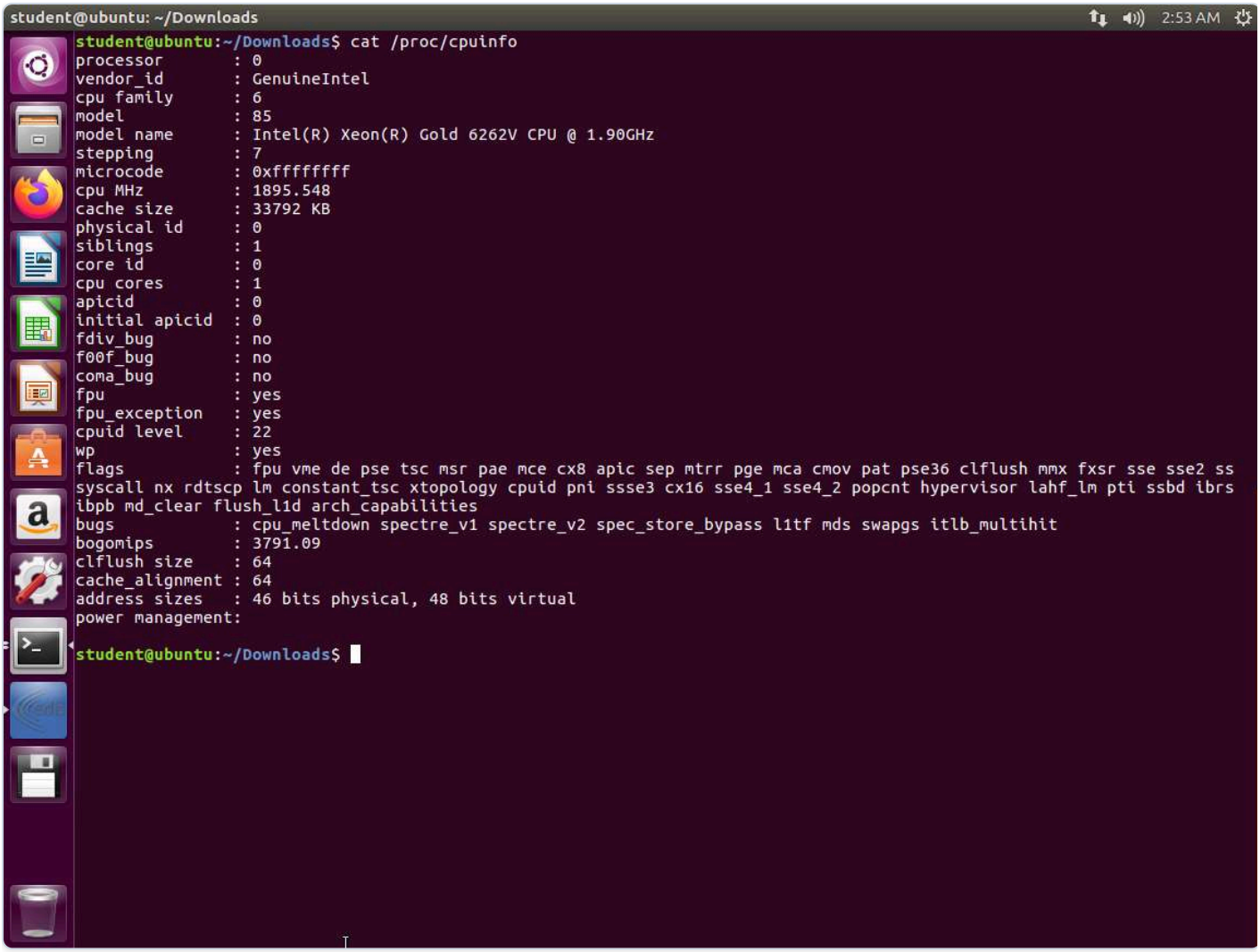
1. Run `lscpu` to inspect processor characteristics:

   lscpu



1. Display CPU info hardware details:

   cat /proc/cpuinfo



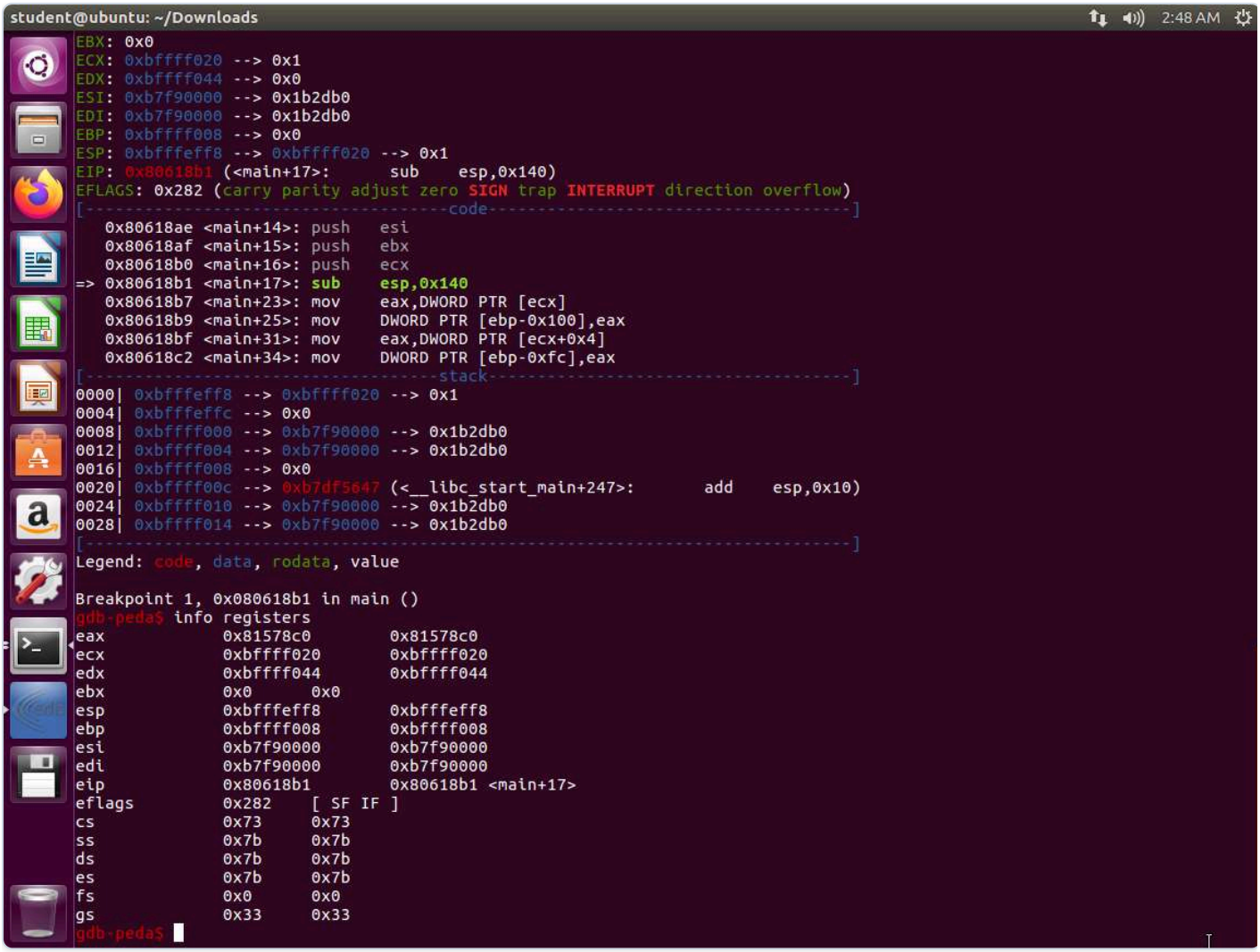
1. Launch GDB on `/bin/bash`, set breakpoint at main, run, and display registers to verify Little-Endian byte order:

   gdb -q /bin/bash

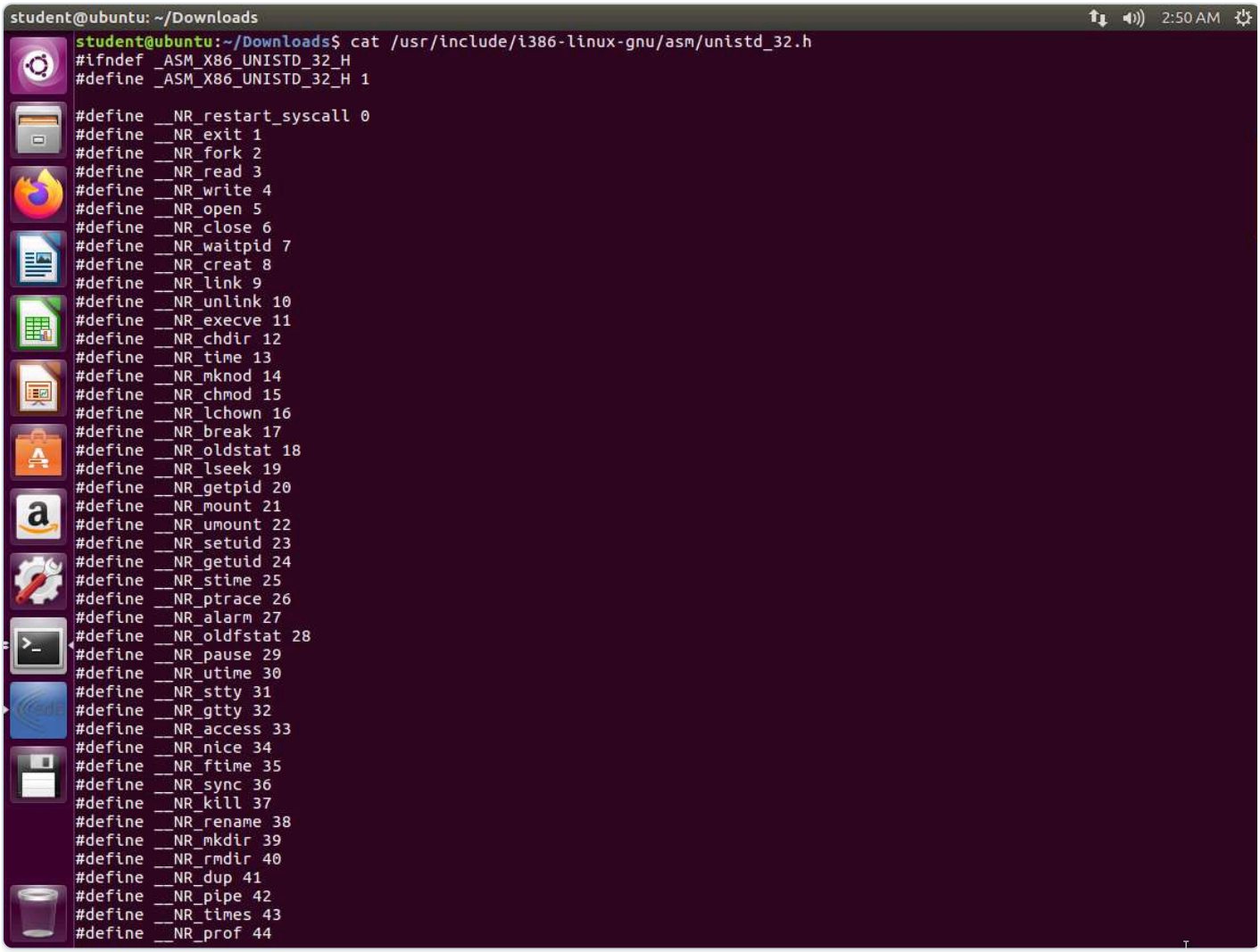
   break main

   run

   info registers

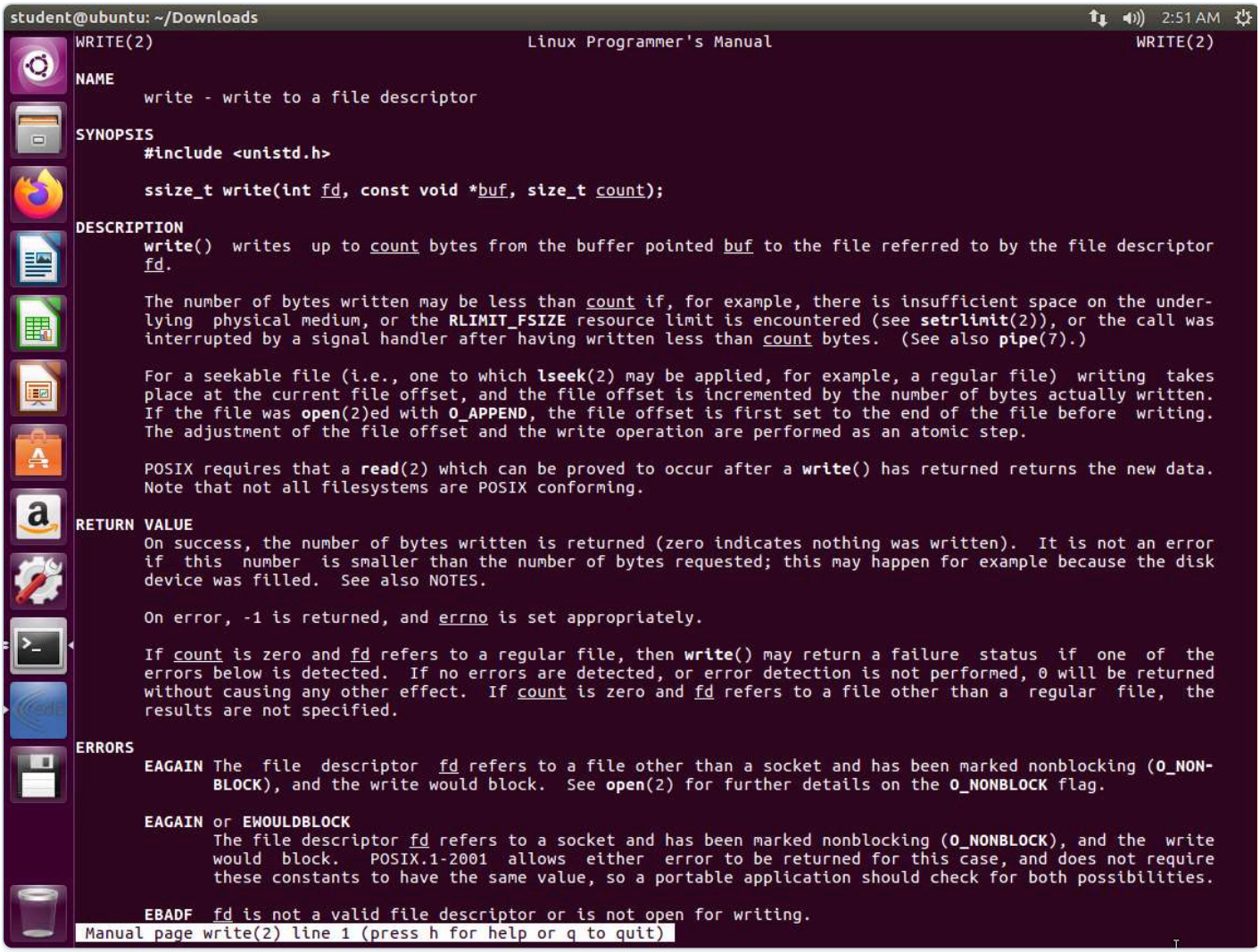


1. Inspect 32-bit x86 system call definitions header file:
   cat /usr/include/i386-linux-gnu/asm/unistd_32.h



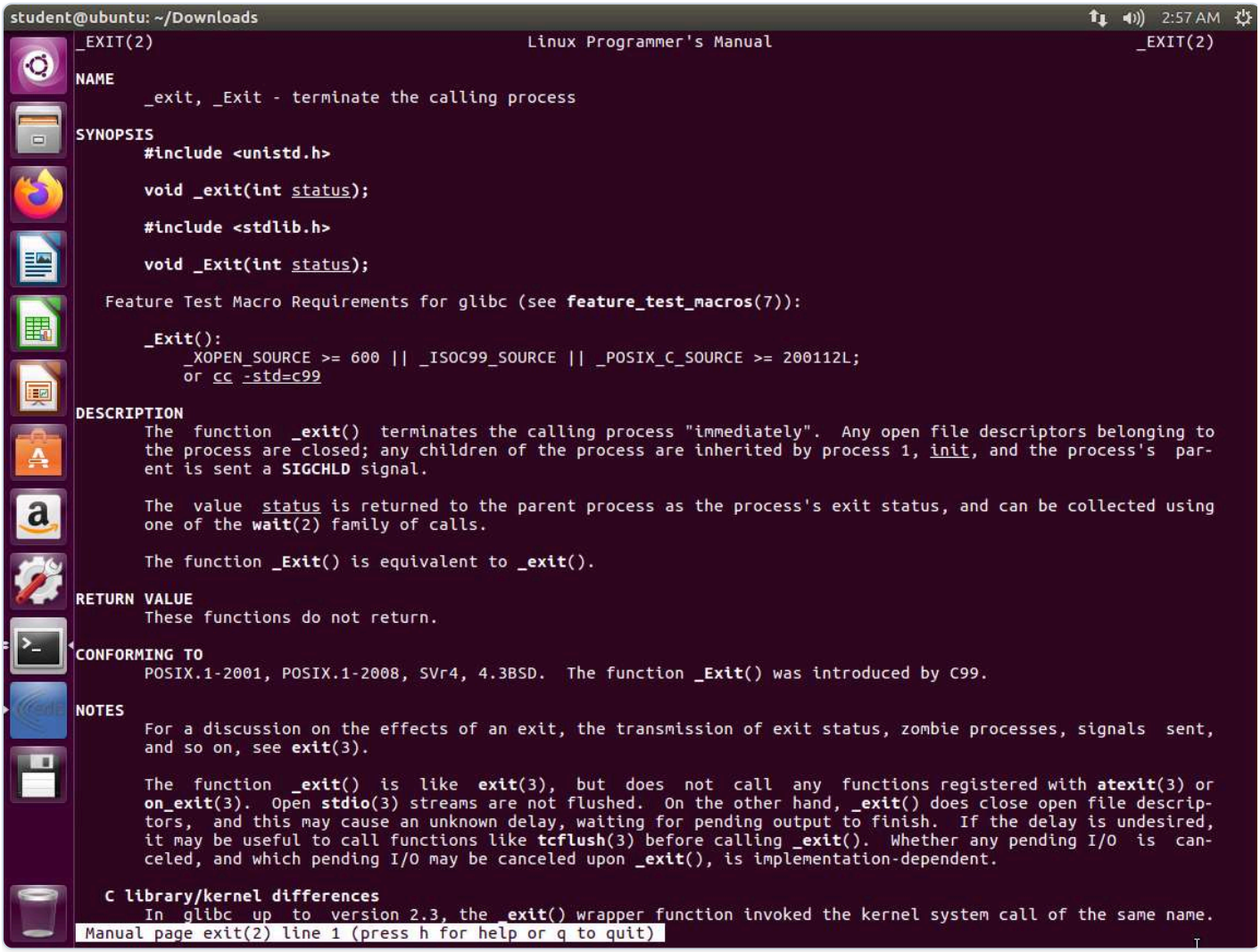
1. View system call manual page for `sys_write`:

   man 2 write



1. View system call manual page for `sys_exit`:

   man 2 exit



1. Create custom assembly source file **code-one.asm**:

```
global _start

section .text

_start:
    ; write(int fd, const void *buf, size_t count)
    xor     eax,eax
    xor     ebx,ebx
    xor     ecx,ecx
    xor     edx,edx
    mov     al,0x4
    inc     bl
    push    0x000a2164
    push    0x6c726f57
    push    0x202c6f6c
    push    0x6c6548
    mov     ecx,esp
    mov     dl,0xf
    int     0x80
```

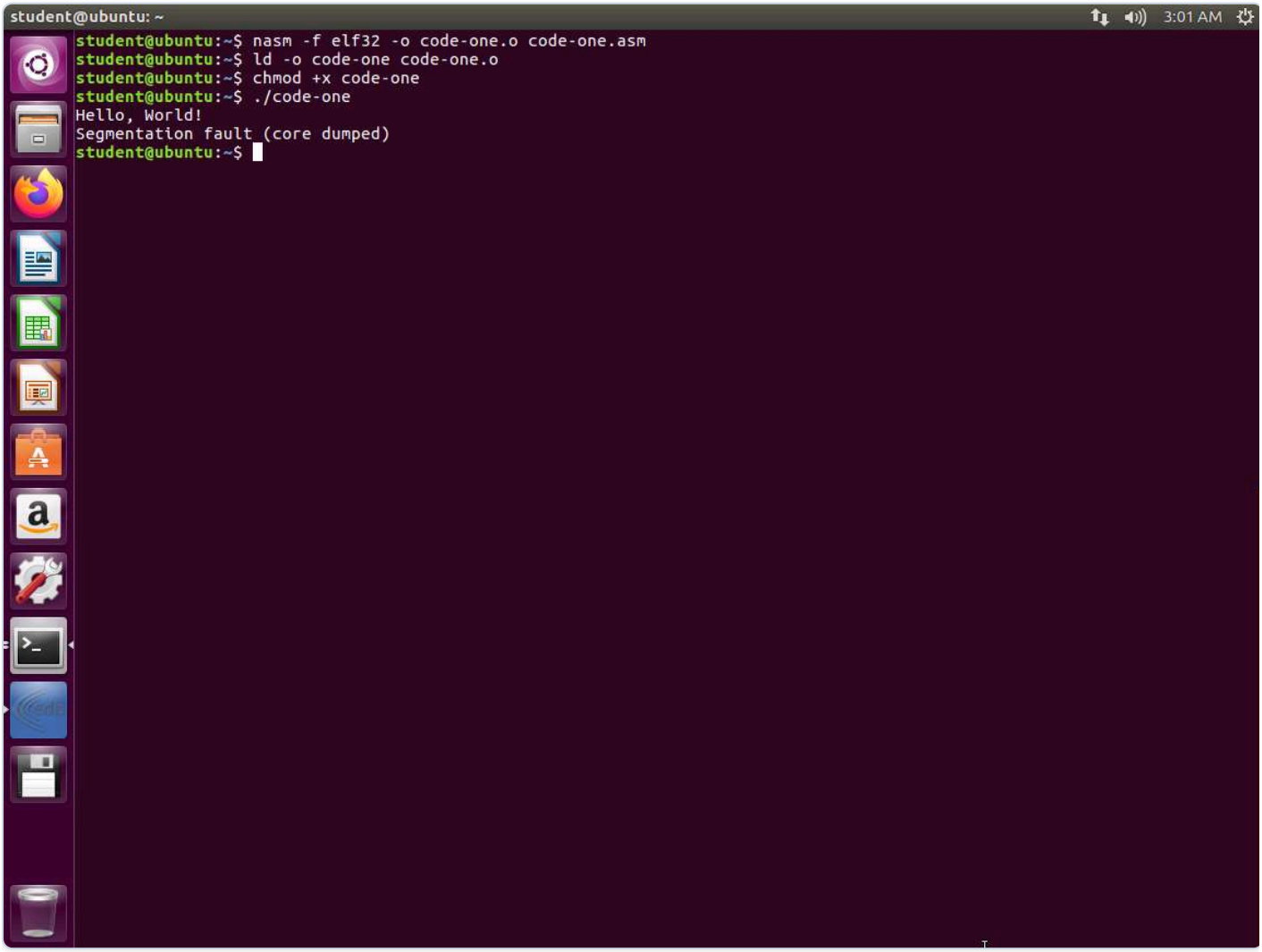
1. Assemble, link, make executable, and run **code-one**:

   nasm -f elf32 -o code-one.o code-one.asm

   ld -o code-one code-one.o

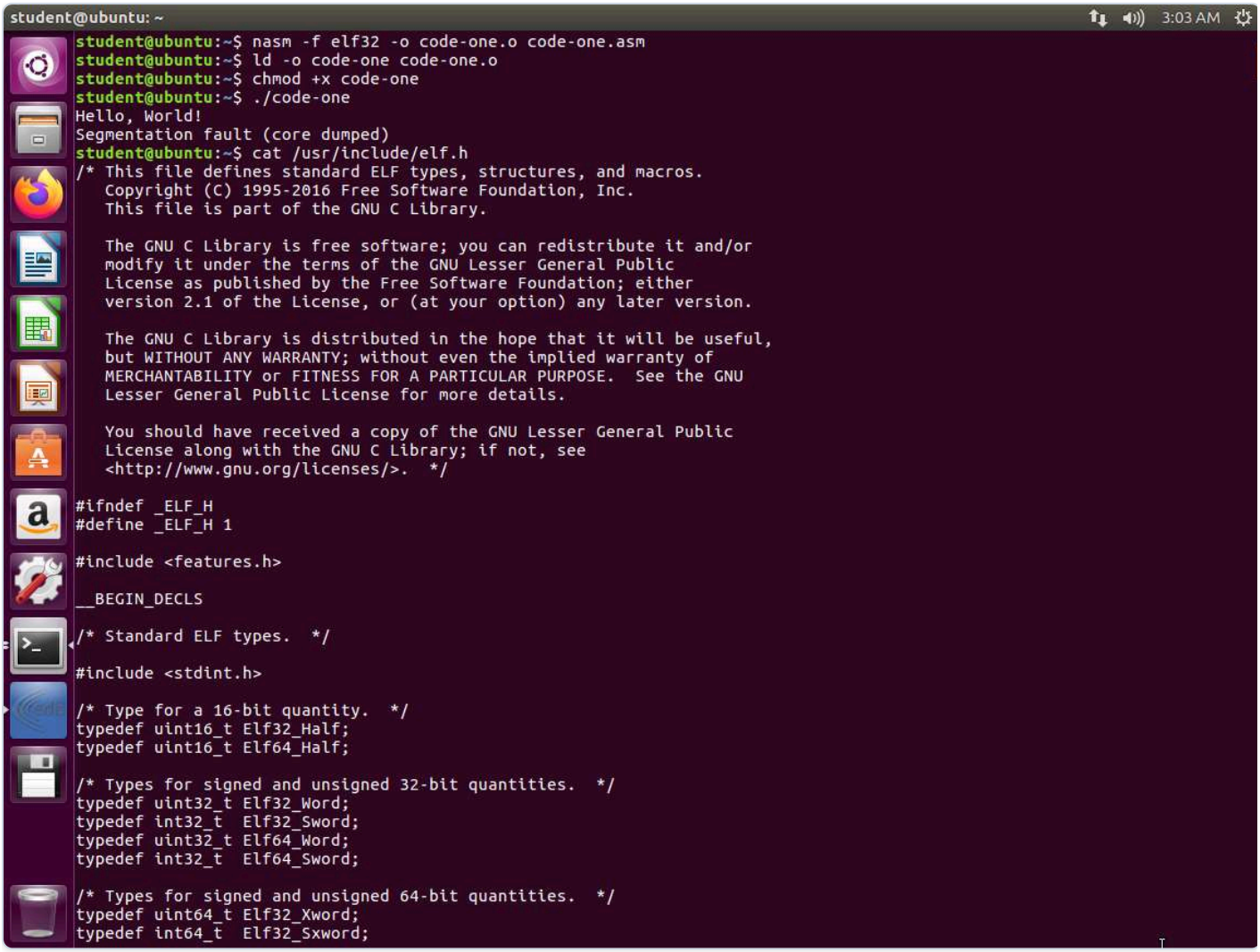
chmod +x code-one

./code-one

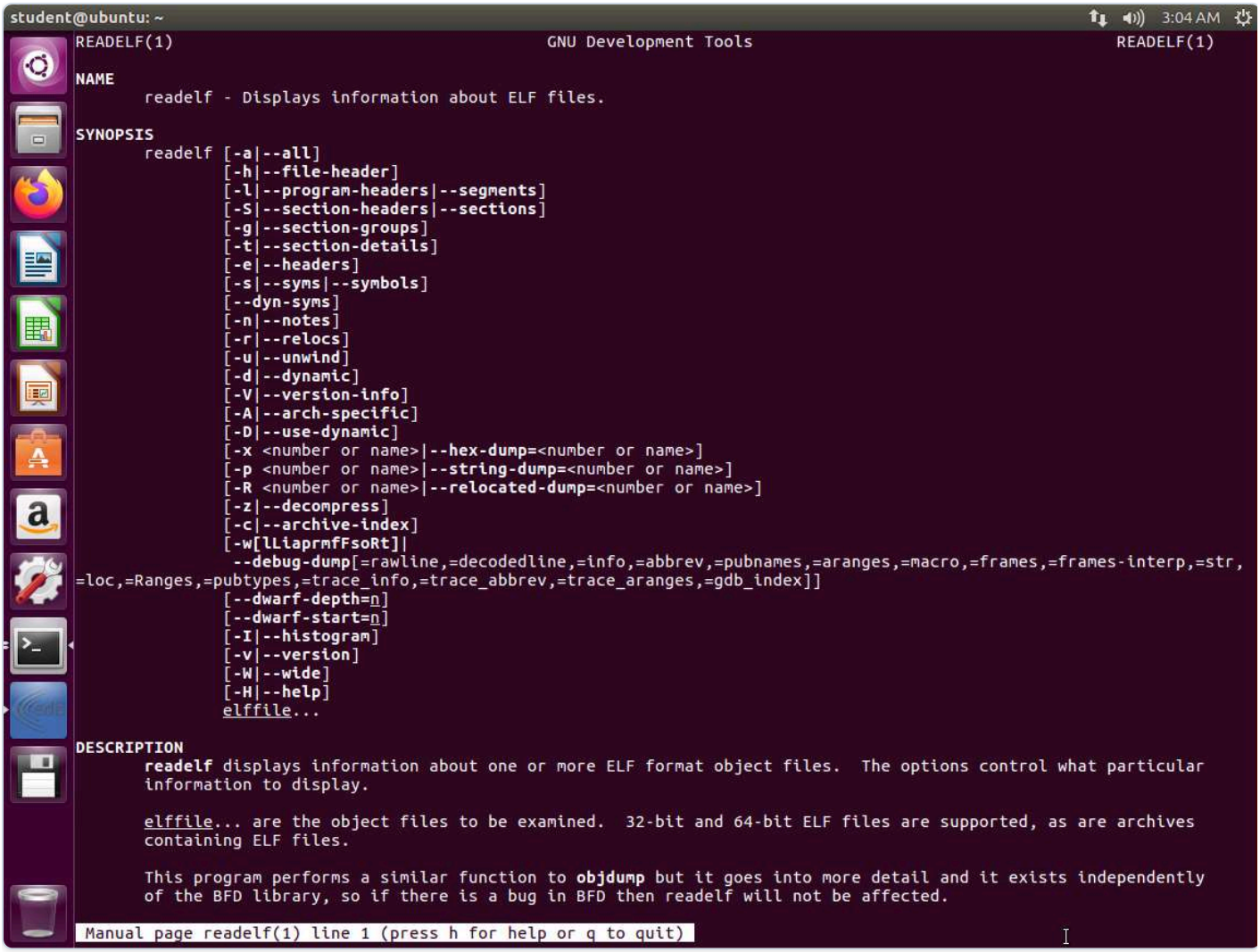


1. Inspect standard Linux ELF header definitions file:

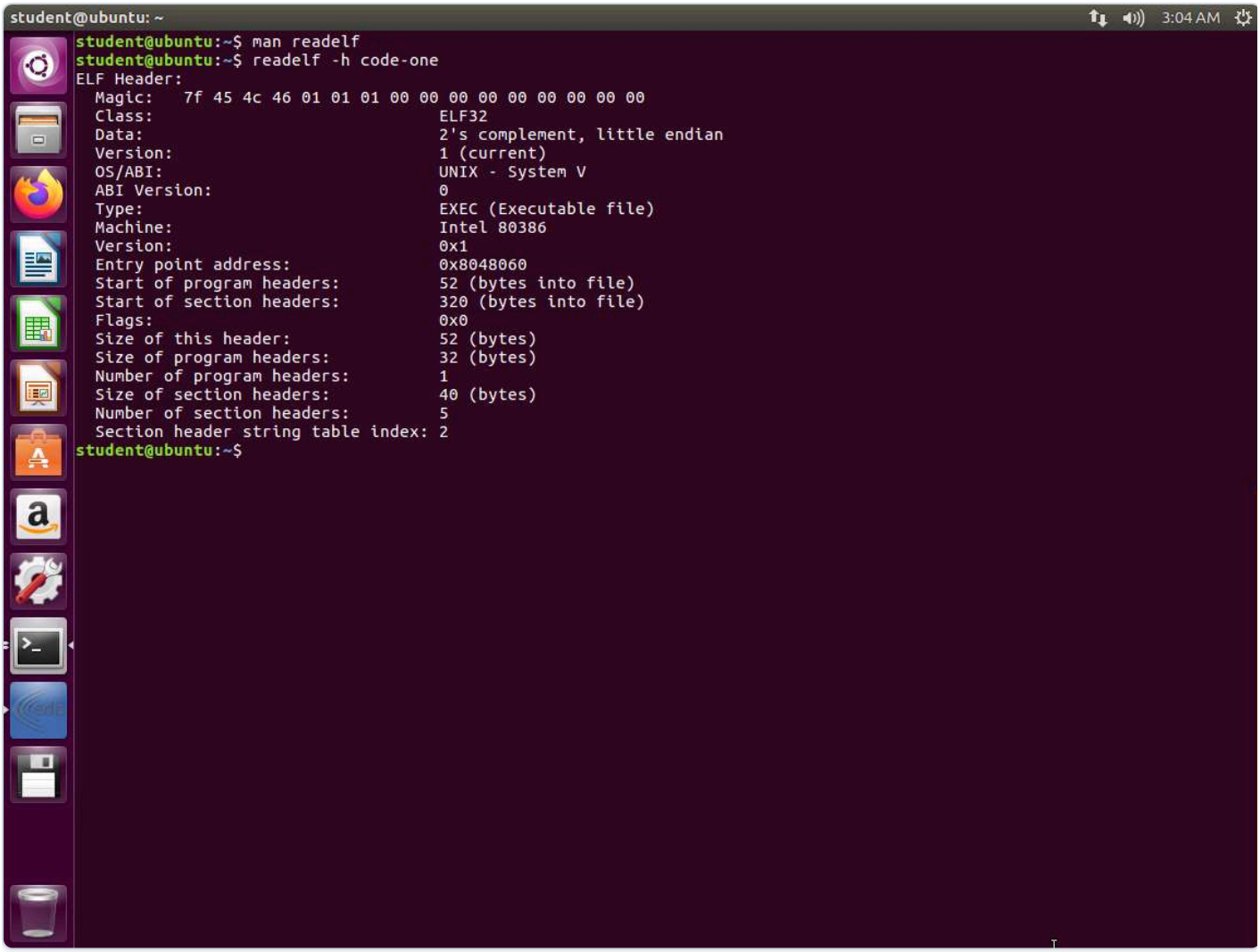
   cat /usr/include/elf.h



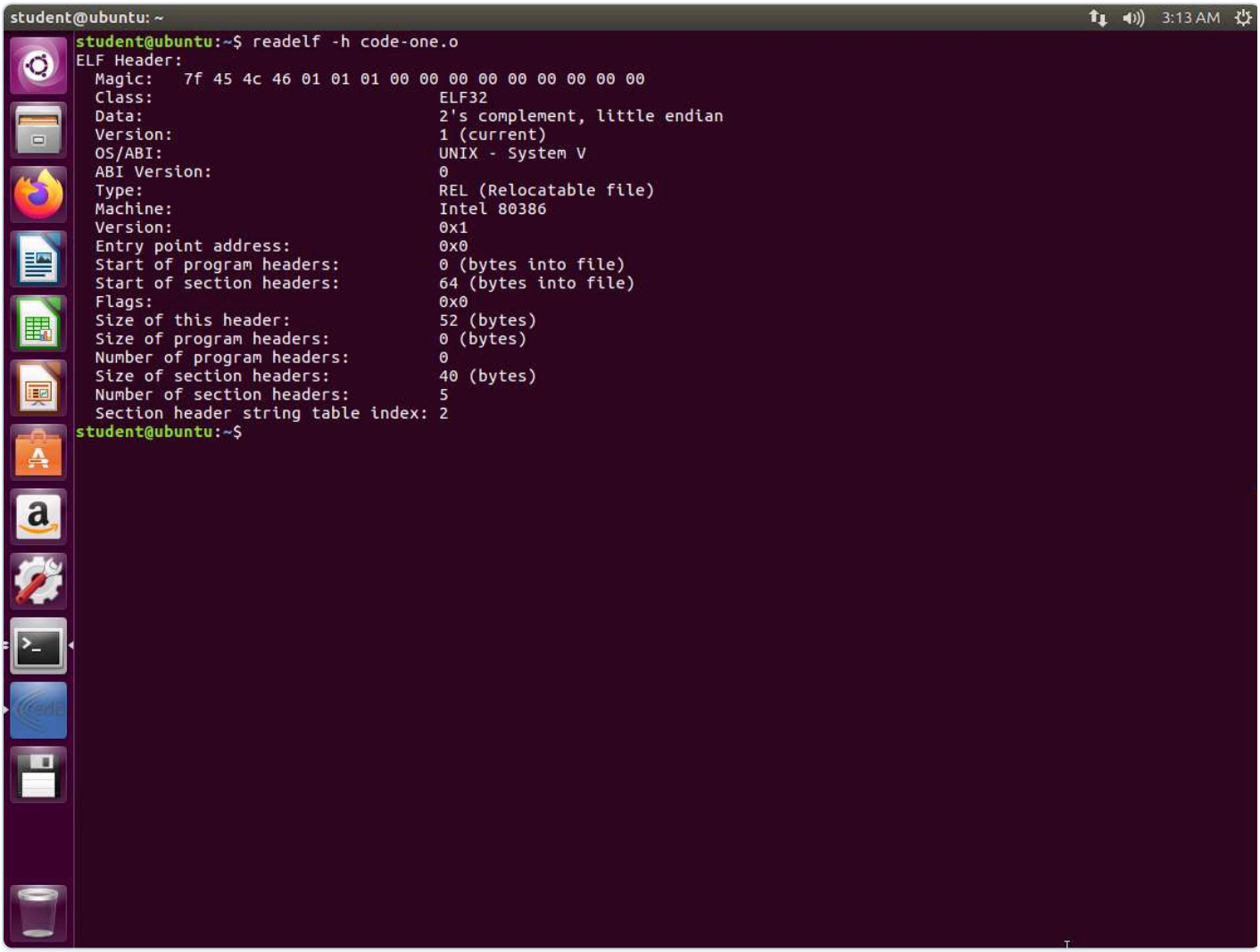
1. Open manual page for `readelf`:
   man readelf



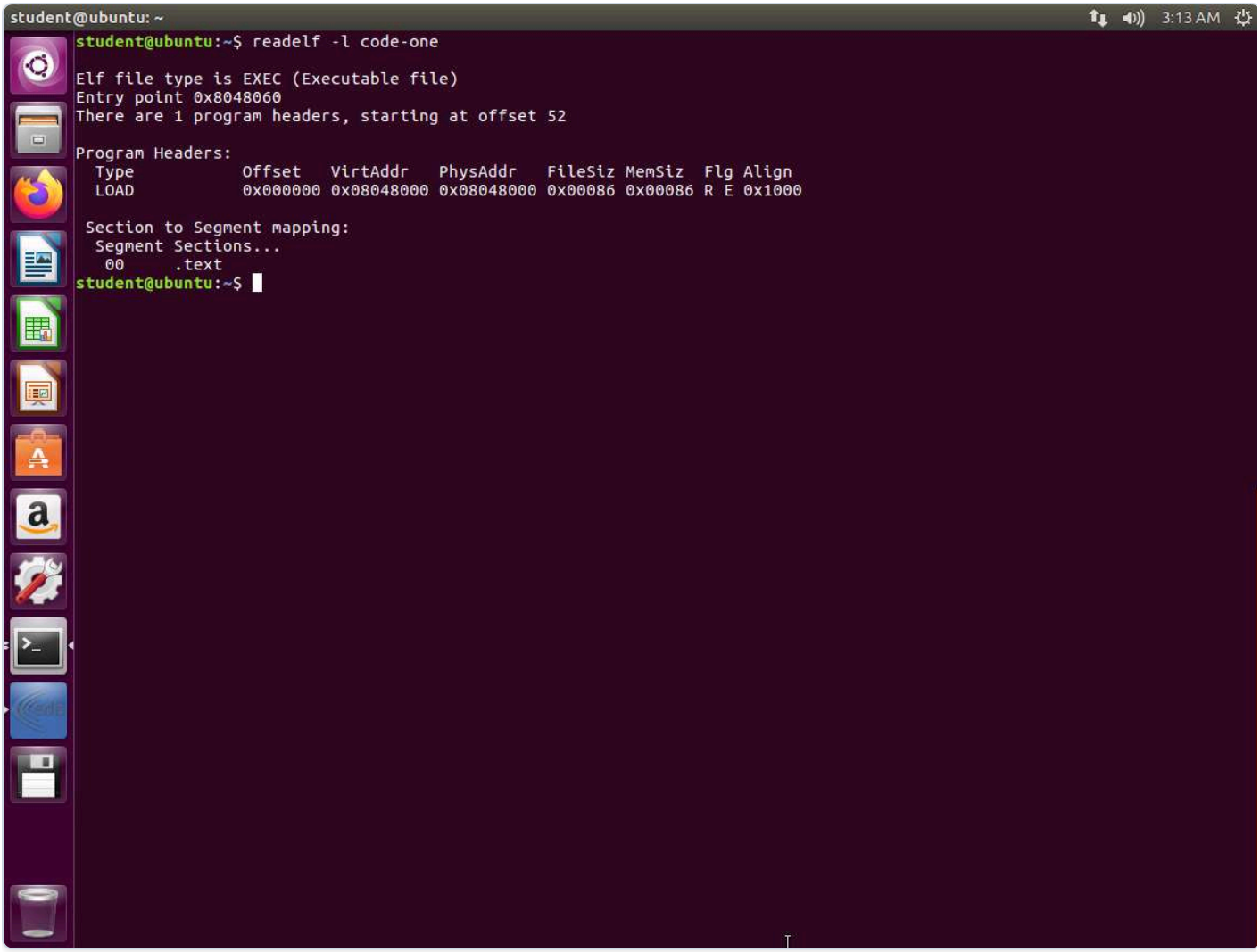
1. Inspect ELF header of compiled binary `code-one`:
   readelf -h code-one



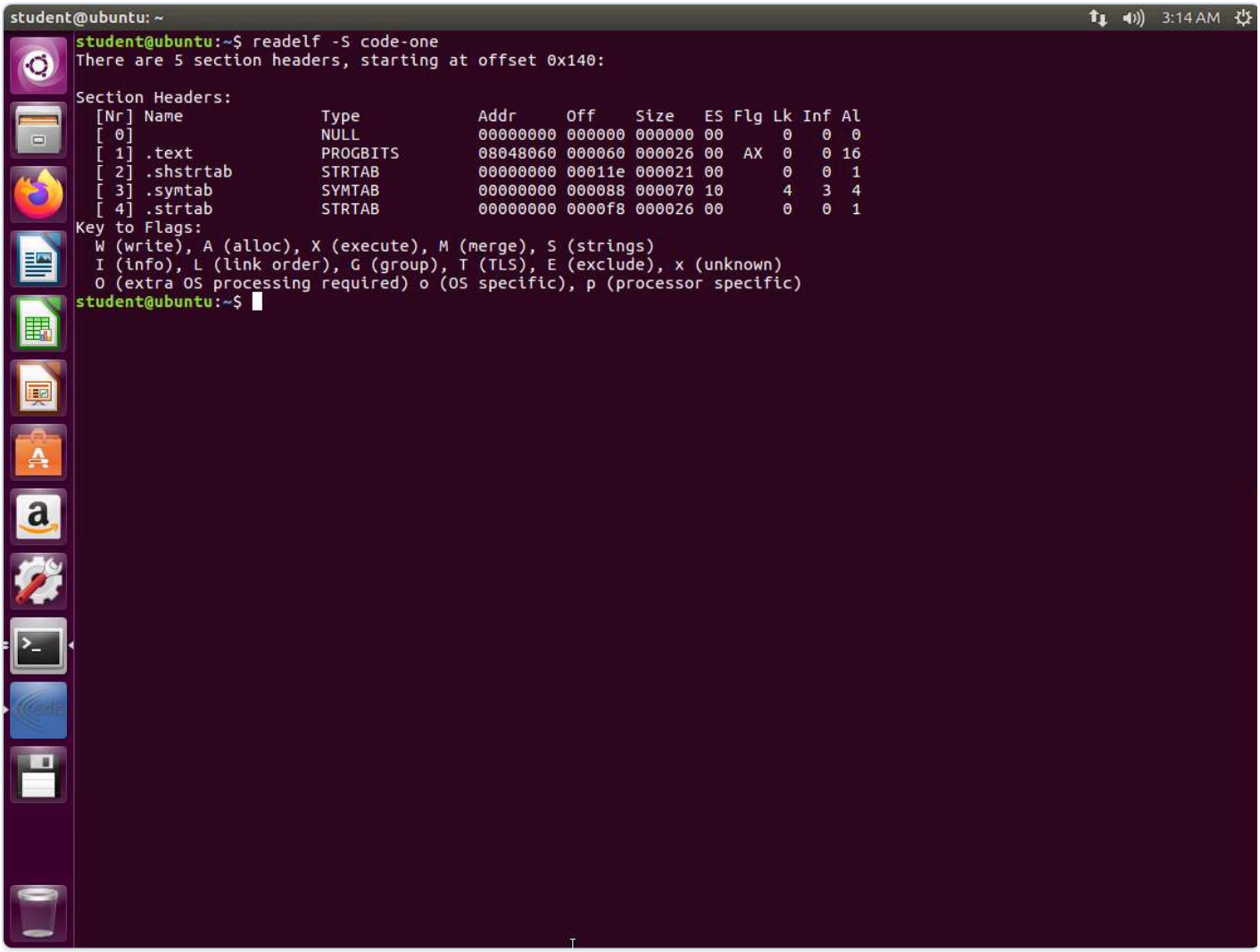
1. Inspect ELF header of object file `code-one.o`:
   readelf -h code-one.o



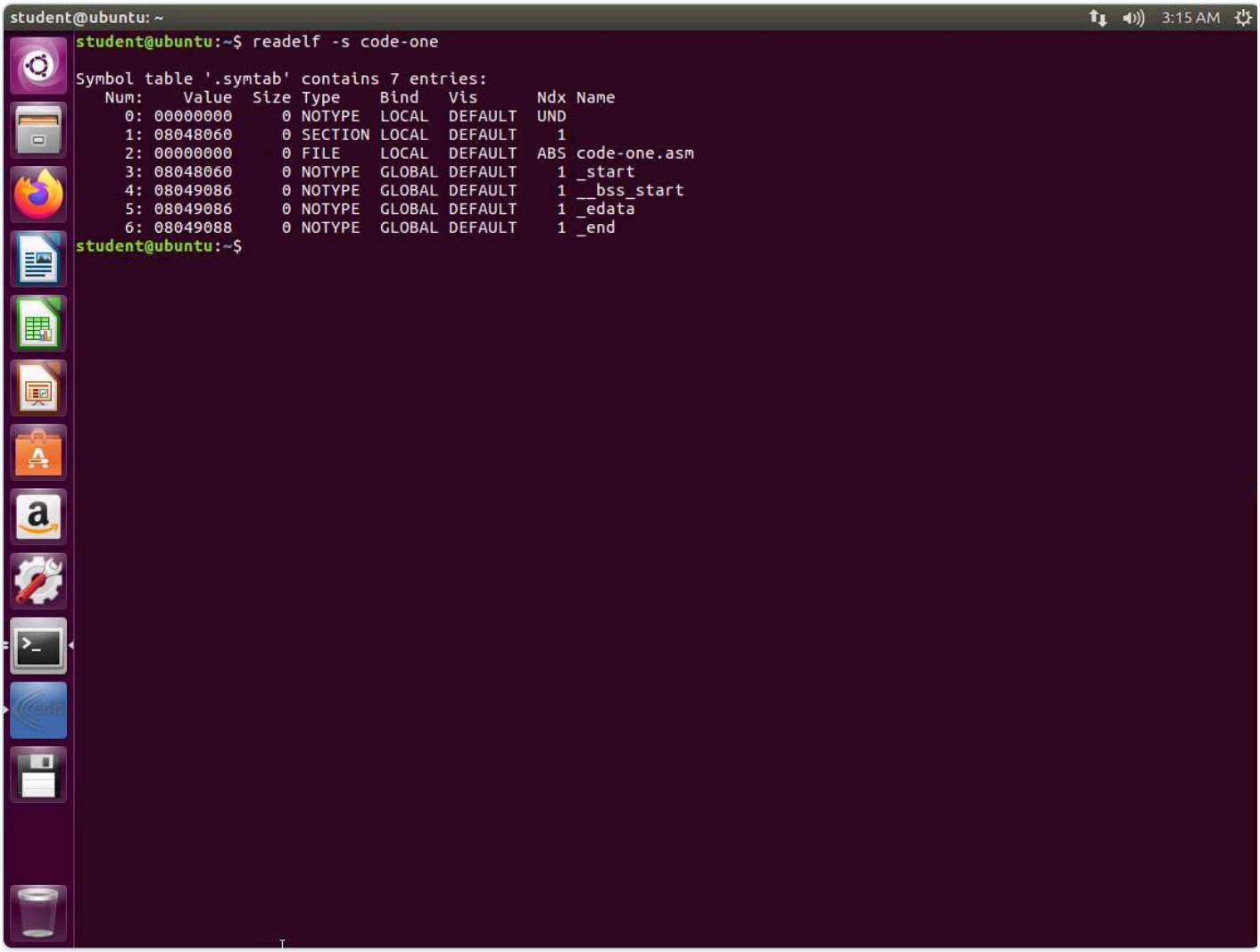
1. Inspect program header segment mappings:
   readelf -l code-one



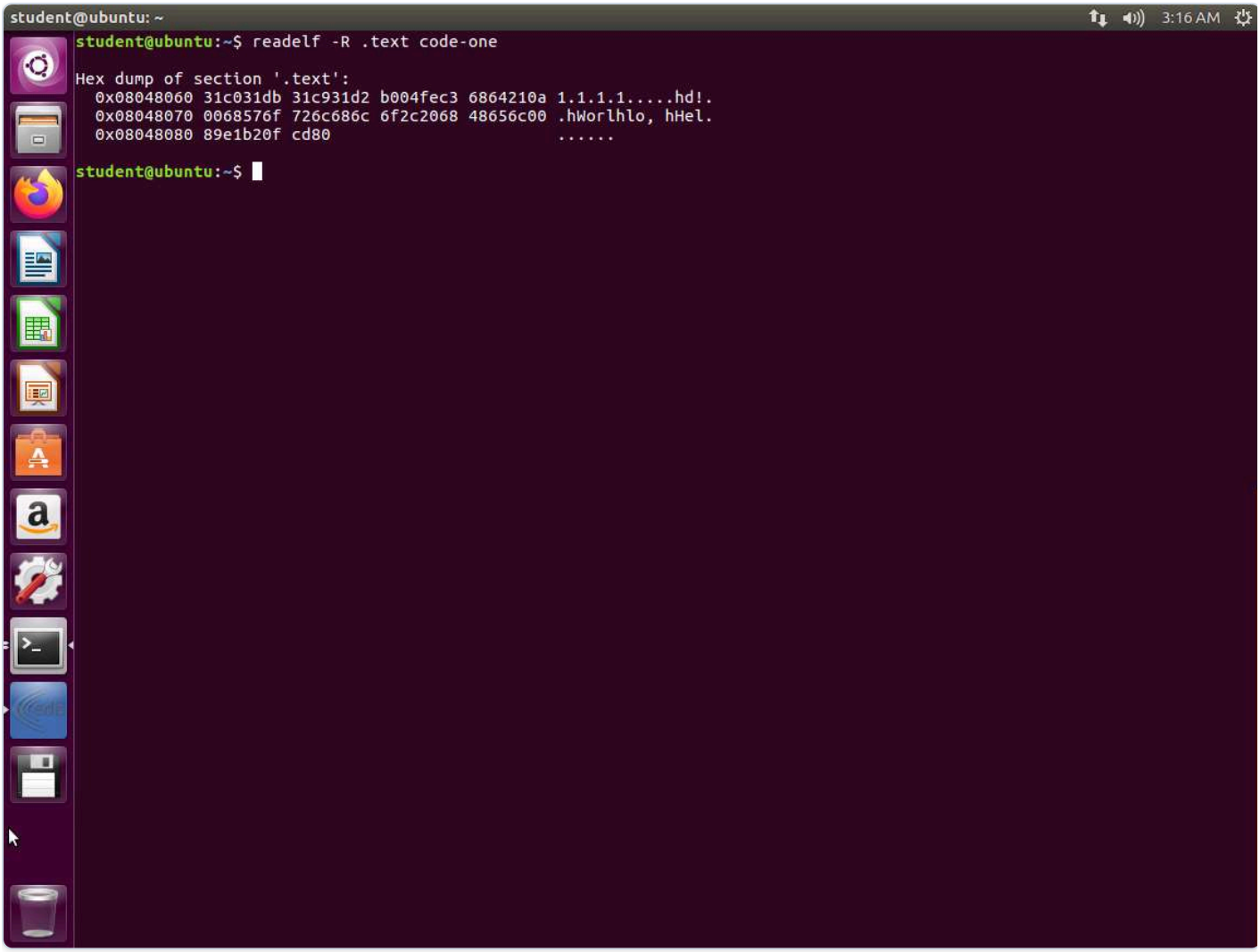
1. Display section headers table:
   readelf -S code-one



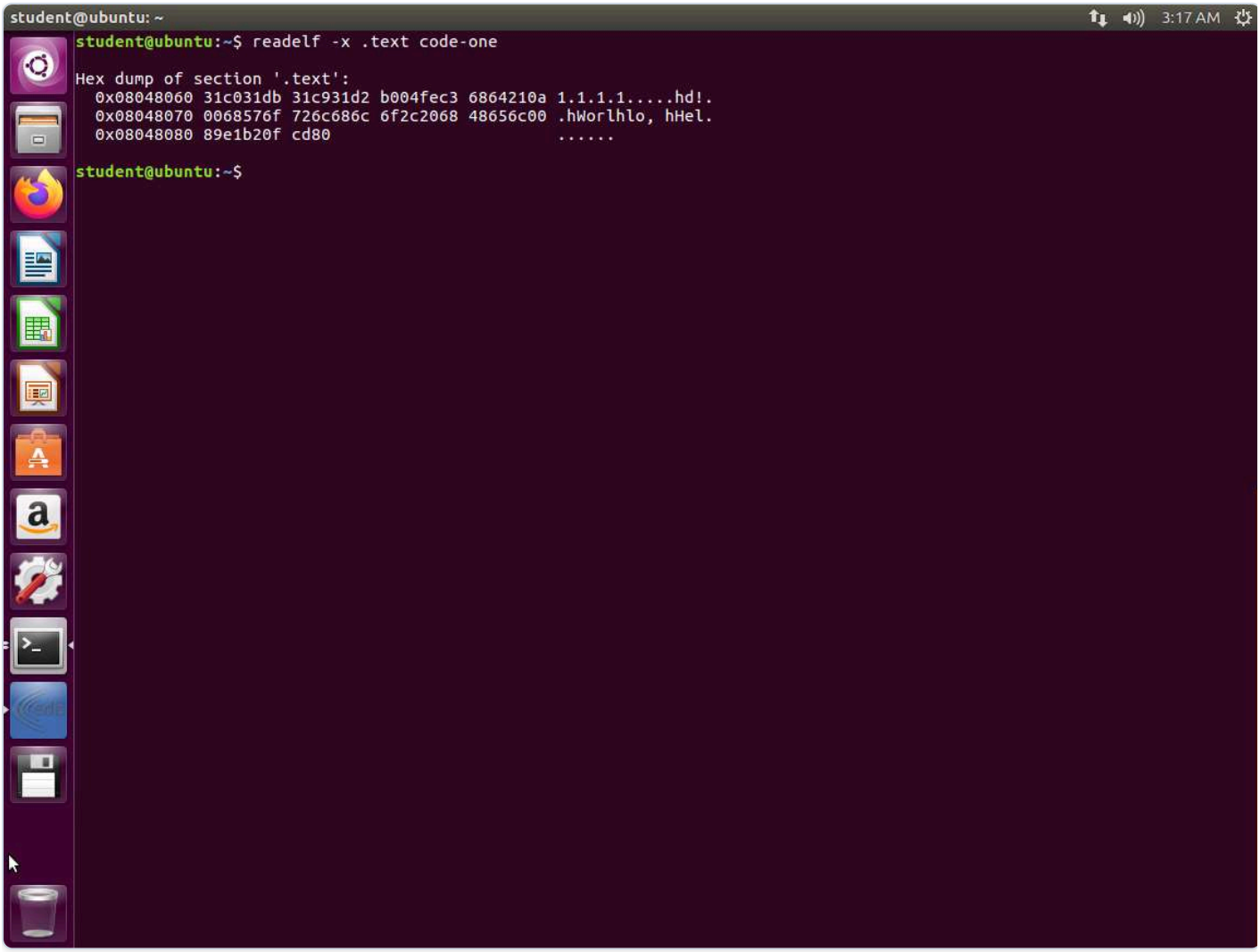
1. Display symbol table entries:
   readelf -s code-one



1. Display relocated bytes for `.text` section:
   readelf -R .text code-one



1. Dump hexadecimal raw bytes for `.text` section:
   readelf -x .text code-one



This concludes the lab exercise.

IMPORTANT: ❓ **Question 11.1.1.1** Conduct a binary analysis of the crackme0x00a file located at /home/student/Downloads on the Target_Software-Test-Linux-32bit machine. Determine the password associated with the file. [!SUCCESS] ✅ **Answer:** **g00dJ0B!**